Requirement Analysis and Specification Document

Best Bike Paths

Bellomo Flavio
Pennino Salvatore
Samarani Anna

Deliverable:	RASD
Title:	Requirement Analysis and Specification Document
Authors:	Bellomo Flavio, Pennino Salvatore, Samarani Anna
Version:	1.0
Date:	07-November-2025
Download page:	https://github.com/flavioblm/BellomoPenninoSamarani.git
Copyright:	Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

Contents

Table of Contents 3

List of Figures 5

1 Introduction 6
1.1 Purpose 6
1.1.1 Goals 6
1.2 Scope 6
1.2.1 World Phenomena 7
1.2.2 Shared Phenomena 7
1.3 Definitions, Acronyms, Abbreviations 7
1.3.1 Definitions 7
1.3.2 Acronyms 8
1.3.3 Abbreviations 8
1.4 Revision History 8
1.5 Reference Documents 8
1.6 Document Structure 8

2 Overall Description 9
2.1 Product perspective 9
2.1.1 Scenarios 9
2.1.2 Domain Class Diagrams 10
2.1.3 State Diagrams 11
2.2 Product Functions 11
2.2.1 Sing-up and login 11
2.2.2 Route creation 12
2.2.3 Path creation 12
2.2.4 Manage inventory 12
2.2.5 Route publication 12
2.2.6 Route planning 12
2.2.7 Scoring and Ranking 12
2.3 User characteristics 13
2.3.1 Guest 13
2.3.2 Cyclist 13
2.3.3 WeatherCondition 13
2.4 Assumptions, dependencies and constraints 13
2.4.1 Domain Assumptions 13

3 Specific Requirements 14
3.1 External Interface Requirements 14
3.1.1 User Interfaces 14
3.1.2 Hardware Interfaces 19
3.1.3 Software Interfaces 19
3.1.4 Communication Interfaces 19
3.2 Functional Requirements 19
3.2.1 Use Case Diagram 20
3.2.2 Use Cases 21
3.2.3 Sequence Diagrams 33
3.2.4 Activity Diagram 47

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.2.5 Requirement Mapping . . . 48
3.3 Performance Requirements . . . 51
3.4 Design Constraints . . . 51
3.4.1 Standards compliance . . . 51
3.4.2 Hardware limitations . . . 51
3.4.3 Other constraints . . . 51
3.5 Software System Attribute . . . 51
3.5.1 Reliability . . . 51
3.5.2 Availability . . . 51
3.5.3 Security . . . 51
3.5.4 Maintainability . . . 52
3.5.5 Portability . . . 52

4 Formal Analysis Using Alloy . . . 53
4.1 Static Analysis With A Complete Model . . . 53
4.2 Static Analysis With A Simplified Model . . . 56
4.3 Dynamic Analysis . . . 58
4.4 Conclusions from Alloy Model Analysis . . . 63

5 Effort Spent . . . 64

6 Reference . . . 65

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

List of Figures

1 Domain Class Diagram 10
2 State Diagram - Route creation 11
3 Screens of *Homepage* 14
4 Screen of *Navigation* 15
5 Screens of the Authentication pages: *Login* and *SignUp* 15
6 Screens of *Account* and *Edit Account* 16
7 Screens of *Inventory* and *Path's details* 17
8 Screens of *Create Route* and *Create Route (Automatic Mode)* 18
9 Screens of *Create Route (Manual Mode)* and *Route Details* 18
10 Use Case Diagram. 20
11 Sequence Diagram related to UC1 - Create Account 33
12 Sequence Diagram related to UC2 - Login 34
13 Sequence Diagram related to UC3 - Guest Access 35
14 Sequence Diagram related to UC4 - Logout 35
15 Sequence Diagram related to UC5 - Manage Account 36
16 Sequence Diagram related to UC6 - Delete Account 37
17 Sequence Diagram related to UC7 - Create Route (Manual Mode) 38
18 Sequence Diagram related to UC8 - Create Route (Automatic Mode) 39
19 Sequence Diagram related to UC9 - Add Route's Details 40
20 Sequence Diagram related to UC10 - Browse Inventory 41
21 Sequence Diagram related to UC11 - Path Browsing 41
22 Sequence Diagram related to UC12 - Path Navigation 42
23 Sequence Diagram related to UC13 - Publish a route 43
24 Sequence Diagram related to UC14 - Unpublish a route 44
25 Sequence Diagram related to UC15 - Path Evaluation 45
26 Sequence Diagram related to UC16 - Delete a route 46
27 Activity Diagram related to UC16 - Delete a route 47

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

1 Introduction

1.1 Purpose

The increasing focus on sustainability and eco-friendly mobility has encouraged many people to use bikes to move around the city, both for daily trips and for sport. However, traditional navigation systems are not designed specifically for cyclists. They are mainly optimized for the shortest distance or travel time and do not show important information for a bike trip, such as road surface quality, typical car speed on those streets, or the presence of obstacles and potholes, which are essential for a cyclist who wants to plan a safe and comfortable route. Cyclists, especially those with sports goals, also need a way to keep an inventory of their past routes and progress. The purpose of Best Bike Paths (BBP) is to provide cyclists a place where they can find, rate, and share good bike paths and manage their own. The system records trips, collects sensor and weather condition data, and lets users add details about the status and of paths. Users can publish their tracks, explore and follow routes made by others, and in this way a community of cyclists is created. Among the shared routes, the application highlights and suggests those that are considered the most effective and the best rated by the community. In this way, BBP helps make cycling safer, more informed, and more enjoyable for its community.

1.1.1 Goals

G1: Cyclists can manually create bike routes or alternatively be assisted by the system to automatically track the paths thanks to the mobile device's sensors. Subsequently, the paths can be reviewed and enriched with general information and with a status before being saved inside the inventory.

G2: Cyclists can visualize and manage their own inventory and search for their saved routes with their statistics, status and meteorological information provided by external services.

G3: Cyclists can publish their routes, sharing them with the community, creating a social component in the system where users can discover different paths proposed by others.

G4: Given a starting point and a destination, the software suggests the best paths to the user visualizing them in a ranking based on scores and allowing the user to be guided by a navigation system along the selected path.

1.2 Scope

The core function of BBP is to support cyclists in recording, managing, and sharing bike paths, and to help all users find safe and suitable routes between two locations. The final user of the software is a cyclist that interacts with BBP through a personal account (not mandatory) on a mobile application. They can create bike paths manually or let the system track them automatically using GPS and sensors on their mobile device. After a trip, they can review the recorded path, confirm or correct detected problems, and enrich it with information on status and other details before saving the route in their personal inventory. BBP stores each route together with statistics such as distance, time, average speed, and enriches these trips with meteorological data obtained from external weather services. Registered users can publish some of their routes, making them visible to the community and contributing to the scores of the paths. Cyclists view public routes, see their status and obstacles, and discover paths shared by the community. In this way, the system introduces a social component where cyclists can explore different paths. Given a starting point and destination, BBP searches inside its archive of recorded and published paths and suggests paths to the user, providing also a navigation system to be followed along the ride. The system ranks the candidate paths using scores based on their status and their average time. BBP focuses on collecting, aggregating, and presenting information about bike paths among a community of users.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

1.2.1 World Phenomena

WP1: Cyclists ride along different bike paths.
WP2: Cyclists decide the starting point and the final destination of their bike paths.
WP3: Cyclists check if the path can be ridden by bike.
WP4: Meteorological conditions influence the bike trip.
WP5: The status and obstacles of a path influence the bike trip.
WP6: The status of paths changes over time.
WP7: Cyclists carry mobile devices with their related sensors such as GPS, accelerometer, and gyroscope while cycling.
WP8: Cyclists decide whether to share or not the information regarding their routes.
WP9: Cyclists evaluate the condition of the path.

1.2.2 Shared Phenomena

World Controlled
SP1: Cyclists manually insert the names of the streets that comprise the bike path.
SP2: Cyclists specify general data and the status of the path.
SP3: Cyclists check and eventually correct the automatically acquired information by BBP.
SP4: Cyclists confirm routes data saving it into their inventory.
SP5: Cyclists publish their routes, allowing the community to see them.
SP6: Cyclists view their previous routes from the inventory.
SP7: Cyclists view the routes published by other users
SP8: Cyclists plan their routes selecting starting point and destination thanks to BBP.
SP9: Cyclists use a navigation system offered by BBP, while they ride along existing paths.

Machine Controlled
SP10: BBP acquires the entire route of the cyclist and its status, with GPS, accelerometer, and gyroscope.
SP11: BBP shows the statistics and the map of the selected path to the cyclists.
SP12: BBP suggests to cyclists the best bike paths sorted by their score.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Guest:** The term refers to the unregistered user. More details are provided later in the document.
- **Cyclist:** The term refers to the registered user on the platform. More details are provided later in the document.
- **Path:** A path is a set of streets rideable by the cyclist from a starting point to an end point. It is automatically created when a cyclist records a route for the first time on a path that does not exist. Once a path is created, it behaves as a route's family aggregating all similar routes under the same path.
- **Route:** A route can be manually created by the cyclist or automatically recorded by the system. Each route is related to a specific path and contains additional details that describe the actual performance of the cyclist on that specific path.
- **Path's Statistics:** Metrics that describe a path: average time, average distance, status and score.
- **Route's Statistics:** Metrics that describe a route: date, duration, distance, average speed, weather conditions and status.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **Score:** A metric that measures the quality of a path. It is calculated by the system and is based on the status of the path and how effective the path is in terms of time.

- **Status:** A metric that reflects the quality and condition of the road (e.g., optimal, medium, sufficient, requires maintenance, bad).
- **Tolerance:** It is a parameter for the search that specifies how far the suggested routes can deviate from the selected points, allowing greater flexibility.

1.3.2 Acronyms

- **BBP:** Best Bike Paths

1.3.3 Abbreviations

- **G:** goal
- **WP:** World Phenomenon
- **SP:** Shared Phenomenon
- **D:** Domain Assumption
- **R:** Requirement

1.4 Revision History

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the WEBEEP page of the Software Engineering II course.

1.6 Document Structure

1. **Introduction**: a brief description of the project that follows from the analysis of the specification provided, it highlights the goals of the product and its scope.
2. **Overall Description**: a more detailed description of the product that comprehends also real world scenarios and analyzes the system from a technical point of view.
3. **Specific Requirements**: a detailed list of all the requirements needed in order to satisfy the goals, and their relationship with domain assumptions.
4. **Formal analysis using Alloy**: a formal description of a portion of real world phenomena analyzed through Alloy in order to highlight inconsistencies between assumptions and phenomena previously modeled.
5. **Effort spent**: the time that each member of the group spent to contribute to the realization of this document.
6. **References**: references to documents or tools used for writing this document.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

2 Overall Description

2.1 Product perspective

2.1.1 Scenarios

Sign-up

Cyclist John opens the platform for the first time and clicks on the *Sign Up* option to create a new account. The system shows a form where John inserts his name, his email and chooses a password to authenticate the next times. After that, John clicks the *Sign Up* button and is shown the home page of the platform.

Login

Cyclist John has already registered on the platform and wants to authenticate. He clicks on *Login* option, the system shows a form where John inserts his credentials, and then clicks the button *Login*. After that, the home page of the platform is shown.

Access as a Guest

Guest Bob wants to visualize the paths available on the platform without performing the Login. He opens the platform and clicks on *Continue as Guest*. After that, the home page of the platform is shown and it is the only interactive screen.

Account management

To view his personal account, John has to click the *Account* button. From this screen, he can access all the statistics related to his activities. From the account screen, John can also edit his account by clicking on *Edit Account*.

Visualize the personal inventory

Cyclist John wants to view his inventory, so he clicks the *Inventory* button. This allows the cyclist to view to all his ride performances. By clicking on a specific path, he can then view its detailed statistics and the related routes

Manage a route

After having visualized the inventory, Cyclist John wants to manage his route. He clicks on the route to expand the view and visualize the management buttons.

- Pressing the *Publish* button, John shares the route with the community. As a consequence, the same button when pressed becomes the dual version *Unpublish*.
- Pressing the *Delete* button, John removes his route from the related path and from his inventory.

Route creation

Cyclist John clicks the *Create* button to access the Route creation screen.

- **Manual Route creation:** Cyclist John begins the process by clicking the *Start* button in the manual route creation section. He first enters the starting point, then adds all the way points, meaning the individual roads he travels along, and finally specifies the end point. Once this step is completed, he proceeds to the next screen, where he provides a description, the duration, and the status. To finish, he saves the route by clicking the *Save* button.
- **Automatic Route creation:** Cyclist John begins by clicking the *Start tracking* button in the automatic route creation section. Then he visualizes a new screen, where he clicks *Start tracking* again. From that moment, the system starts to automatically track the cyclist movements. Once the ride is completed, John stops the tracking, reviews the automatically generated route, and finalizes by saving it.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

Route Planning

To plan a new route, the cyclist John clicks the *Home* button and enters the starting point and the destination of the path he intends to follow. John may also choose to specify a tolerance in kilometers. He then presses *Search paths* to view all the routes that match his criteria.

Visualize a path from the search results

By selecting a path, John can view the map, the average time, the distance, the status, and the score associated with that path. Moreover, similar routes will be displayed in the section below.

Navigation

User Jade visualizes a path and clicks the button *Start Navigation*.
The platform provides Jade with a navigation map to guide her along the selected path. When the end point is reached, a *Path completed* message is displayed.

- If Jade is a cyclist in the next screen she provides all the details of the routes and proceeds to save the route in his inventory by clicking the *save* button.
- If Jade is a guest the system shows a message "You have to login to save a route".

2.1.2 Domain Class Diagrams

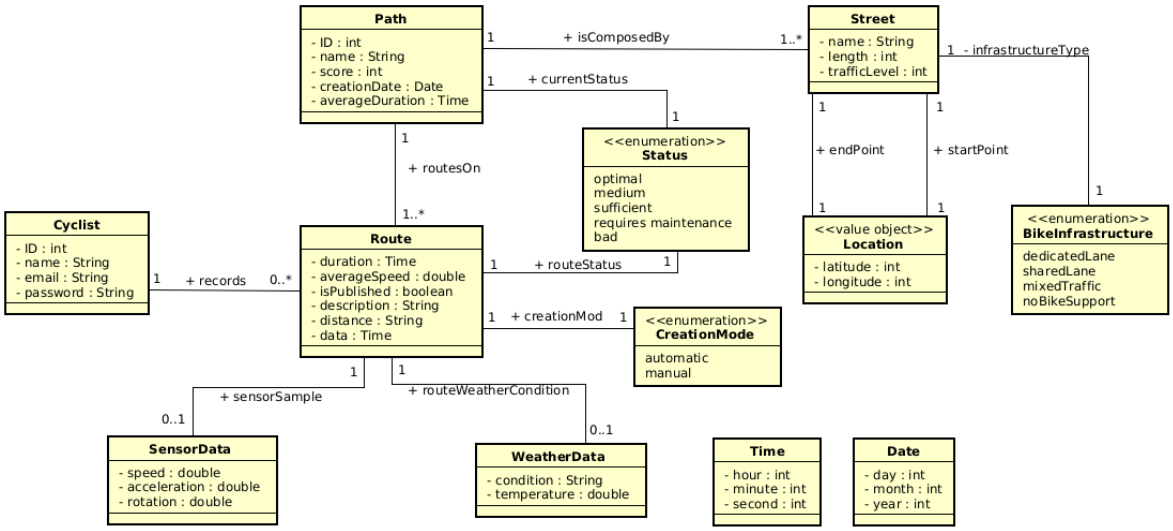


Figure 1: Domain Class Diagram

This is the class diagram of BBP. The system can also be used by guest user but it is not necessary store any information about them because guests can only view content but can not interact with the system by creating and managing their routes. All the other users of the app are registered and they are described by the **Cyclist class**. The classes Path and Route describe related concepts but they have a different role in the application.
Path class, a path is the most general description of a set of streets, **Street class**. The streets of the paths can be collected automatically by the mobile device (and then confirmed by the cyclist) or directly entered manually by the Cyclist, this distinction is modeled by the **CreationMode enum**. For each street it's stored not only general information but also the exact location where it starts and ends **Location valueObject** and the type of bike infrastructure **BikeInfrastructureType enum**.
Route class, a route instead represents an actual ride performed by a cyclist on a specific path. If a route

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

is performed on a path that is not present yet in the system, a new path will be created and that route will be set as the route that generated the path. Each route is enriched with sensors data **SensorData class** and with weather condition, **WeatherCondition class**.
Status enum, the status represents the quality of the road surface when a route is recorded or more broadly the current condition of a path, updated using the most recent information available in the system.

2.1.3 State Diagrams

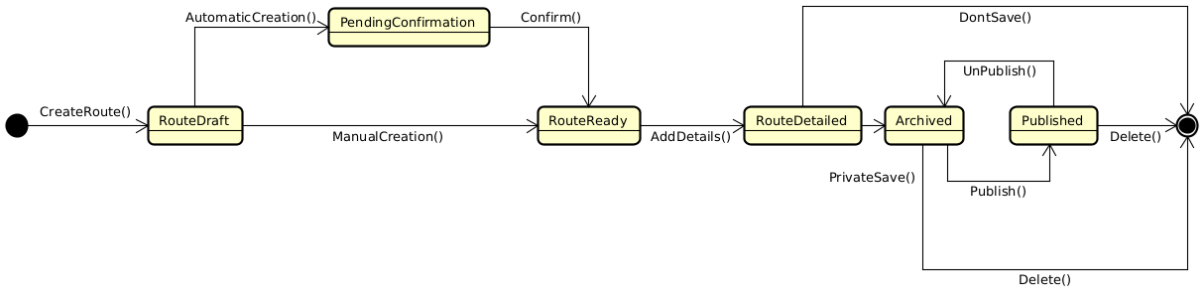


Figure 2: State Diagram - Route creation

The state diagram in figure describes the lifecycle of a route from its creation to its publication, archiving, or deletion. The process starts from the initial state, where the *CreateRoute()* event moves the route into the **RouteDraft** state, representing a newly created draft. From this state, two alternative ways are possible:

- *AutomaticCreation()* leads to the **PendingConfirmation** state until the Cyclist does not confirm the information collected. Once the *Confirm()* event occurs, the state changes into **RouteReady** state
- *ManualCreation()* leads directly to the **RouteReady** state, skipping the confirmation phase

The **RouteReady** state indicates that the route is prepared but not yet complete, because the details are still missing. The *AddDetails()* event makes the transition to the **RouteDetailed** state happen, where all required details have been added. From this state, the route enters a region that manages its visibility and persistence:

- With *PrivateSave()*, the route is stored and goes to the **Archived** state.
- The *DontSave()* event allows the Cyclist to discard the route, leading directly to the end of the lifecycle.
- From **Archived**, the *Publish()* event allows the route to be made public, transitioning it to the **Published** state. Dually, from **Published** state, the *Unpublish()* event moves the route back to the **Archived** state.

Finally, from both the **Archived** and **Published** states, the *Delete()* event transitions the route to the final state, representing its permanent removal.

2.2 Product Functions

2.2.1 Sing-up and login

This feature allows cyclists to access their personal account using the authentication credentials (e-mail and password). In his personal area, the cyclist can manage his account uploading a picture or changing the credential, view the total number of routes characterized by total distance and time. Unregistered users can login as guests and can only view published routes.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

2.2.2 Route creation

Cyclists can create new routes either manually or automatically and upload them to the platform. In manual mode, the user has to specify each road traveled one by one; in automatic mode, instead, they start the recording and the system collects data through the device's GPS and other sensors. Users can also enter and share information regarding the condition of cycling routes, specifying their quality level and reporting the presence of any relevant obstacles, such as potholes or damaged sections. A cycling route refers to a path equipped with a dedicated bike lane or an area where car traffic is limited and speed limits are compatible with the average pace of a bicycle. Each route is later enriched by the system through the addition of various statistics, such as total distance, average speed, and other performance indicators. When available, weather conditions retrieved from external services are also integrated.

2.2.3 Path creation

A path is automatically created by the system when a cyclist records a route for which no corresponding path already exists. Once the path has been created, any new route with a similar path will be associated with that path.

2.2.4 Manage inventory

Cyclists can visualize their inventory filtering by published paths and by the status of the paths. Moreover it is possible to order the paths inside the inventory by: date, distance and score. They can also see a specific route of a path and decide to publish, unpublish or delete it.

2.2.5 Route publication

Cyclists can choose to share one or more of the routes they have created with the community. This operation can be easily performed from their personal inventory, where they can select the routes they wish to make public. Once published, the route becomes visible to other cyclists on the platform and may appear in the results when users explore new paths to follow. In this way, each cyclist contributes to enriching the library of available routes, promoting sharing and the discovery of new experiences.

2.2.6 Route planning

Users can search for new paths to follow within the platform by entering a starting point, a destination, and a tolerance value expressed in kilometers allowing greater flexibility in the search. Once these parameters are provided, the system generates a list of paths that match the given criteria. For each path, the main information is displayed: name, distance, average time, status and score. Users can choose an existence path and start the navigation and if they are Cyclists they may also choose to save the route in their inventory

2.2.7 Scoring and Ranking

The system calculates a score for each path in order to evaluate its overall quality and its ability to satisfy the user's request. The score is determined by combining two main factors: the condition of the path and its effectiveness, meaning how suitable it is for connecting the user's specified starting point and destination. The goal of the score is to allow the system to rank the different paths during searches.
Since path conditions may change over time, to ensure that the score remain accurate and up-to-date, when multiple users report different information about the same path, the system merges them by evaluating the date of the submitted data, giving priority to the most recent and the number of consistent reports, in order to increase the reliability of the classification.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

2.3 User characteristics

There are two types of user that can interact with BBP and the main difference is the ownership of a personal account on the platform. In summary, registered users, called Cyclists, have full access to all features. Unregistered users, referred to as Guests, have limited access to the platform. In this document the term *User* will be used referring to both Cyclist and Guest.
Moreover there is a third actor that is the WeatherCondition service that is invoked by the system and does not interact directly with it.

2.3.1 Guest

The Guest is an unregistered user who can access the platform without creating a personal account. Their interaction with the system is mainly focused on consultation: they can search for and view available bike paths, accessing their statistics, map, and status. The Guest can also benefit from the route planning system.

2.3.2 Cyclist

The Cyclist is a registered user of the platform who owns a personal account. Once logged in, they can actively interact with the system, contributing to the growth of the community.
The Cyclist can access all features available to the Guest, as well as additional ones. In particular, the Cyclist has the possibility to create new paths either manually or automatically. After defining a path, the Cyclist can enrich it with details and evaluate with a status before saving it as a route inside his personal inventory.
The Cyclist therefore plays a central role within the platform: they actively contribute to the creation, evaluation, and publishing new routes, supporting the continuous improvement of the system and the expansion of the archive of paths available to all users.

2.3.3 WeatherCondition

The WeatherCondition is an external service used by the system to enrich routes data with information about the weather condition.

2.4 Assumptions, dependencies and constraints

2.4.1 Domain Assumptions

- **D1:** The cyclist's account data is correct.
- **D2:** The cyclist remembers his credentials.
- **D3:** The cyclist manually adds a route only after actually completing it.
- **D4:** When creating and evaluating a route, the cyclist inputs accurate and realistic information.
- **D5:** Reliable meteorological information.
- **D6:** If cyclists use GPS, accelerometer and gyroscope we assume that they are accurate and work correctly.
- **D7:** There are always paths available from point A to point B for any tollerance level.
- **D8:** The starting and ending point are reasonable, so that they do not necessarily require the use of other means of transportation.
- **D9:** All users have a reliable internet connection.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The images below show the most important screens of the application.

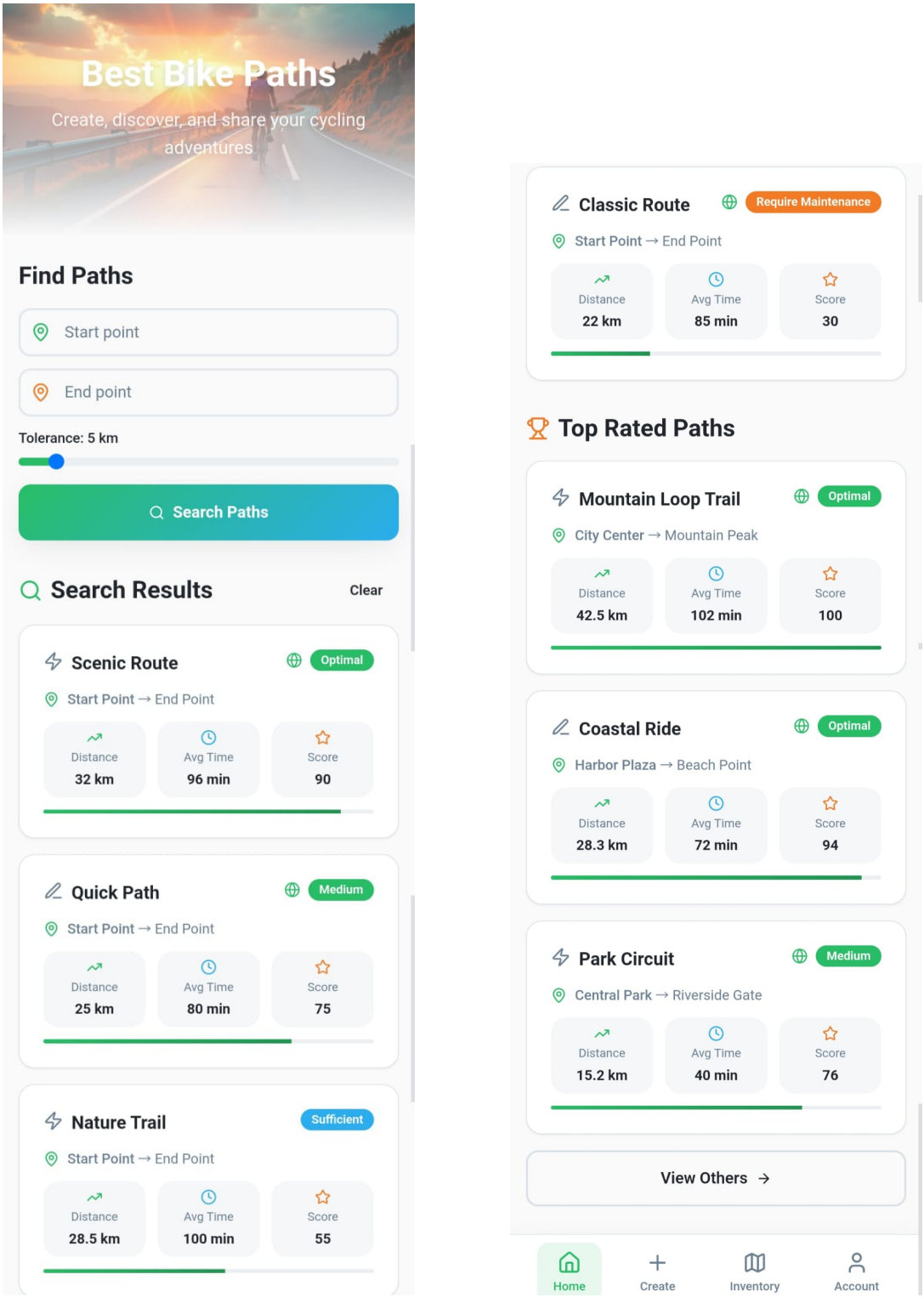


Figure 3: Screens of *Homepage*

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

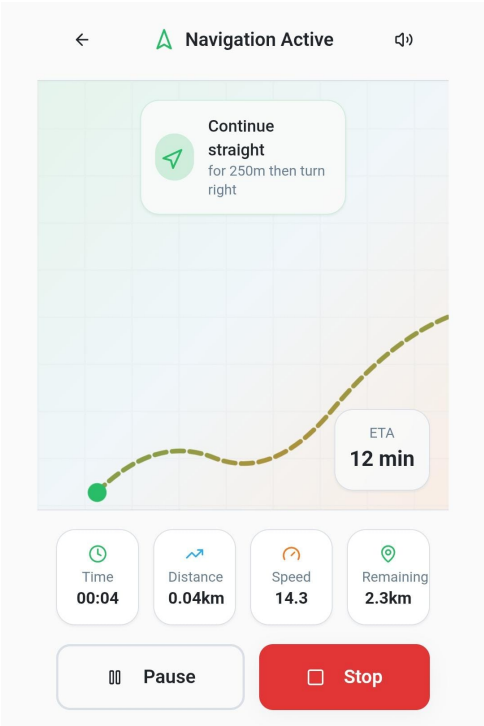


Figure 4: Screen of *Navigation*

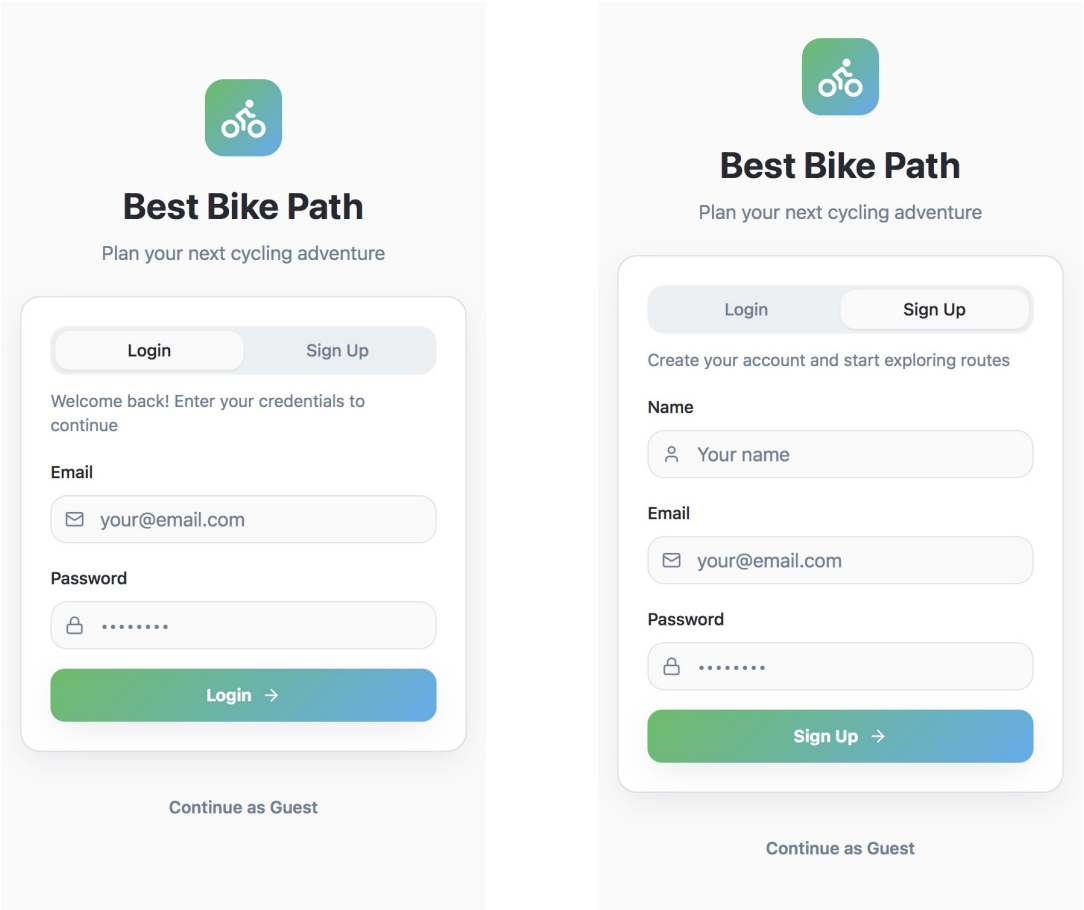


Figure 5: Screens of the Authentication pages: *Login* and *SignUp*

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

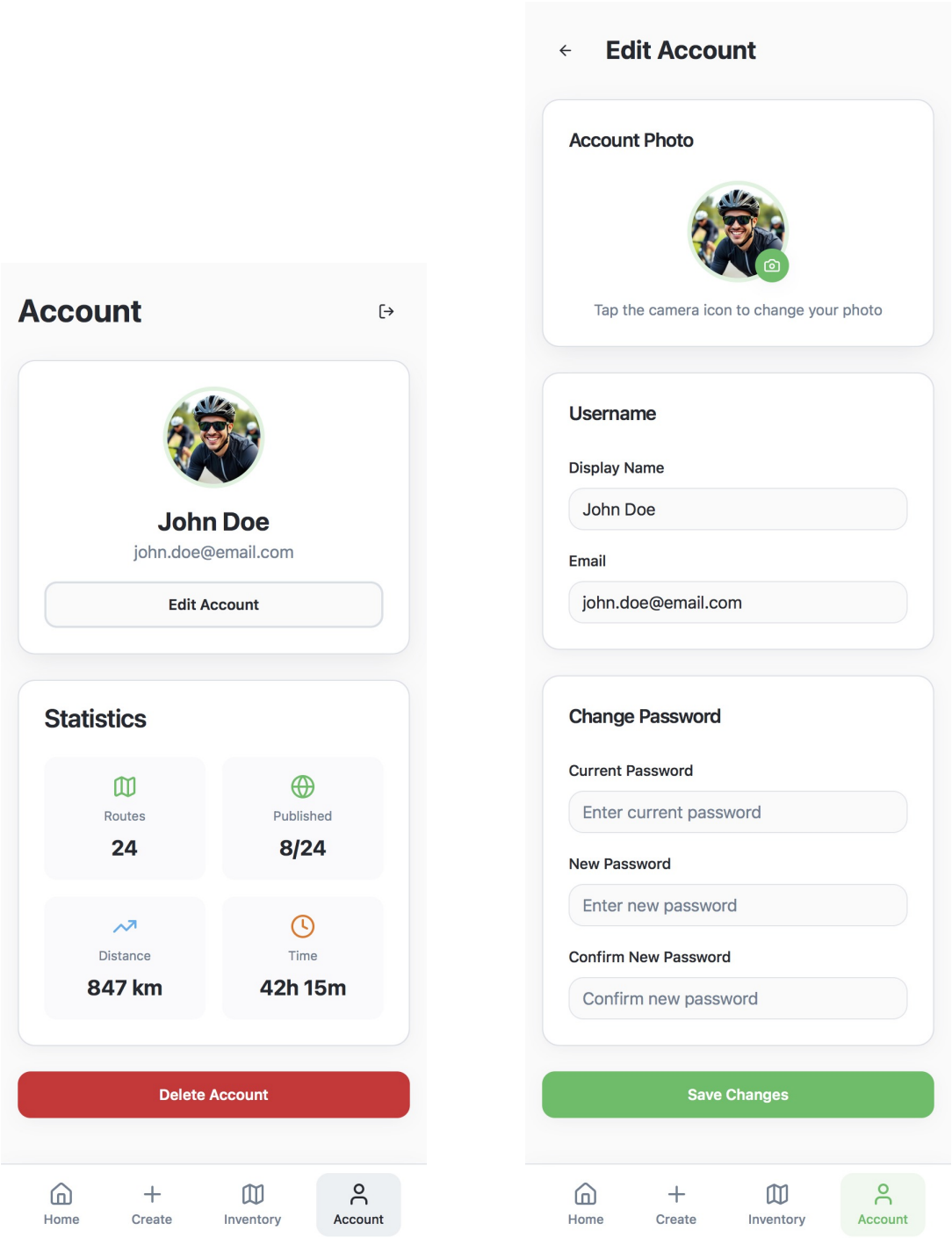


Figure 6: Screens of *Account* and *Edit Account*

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

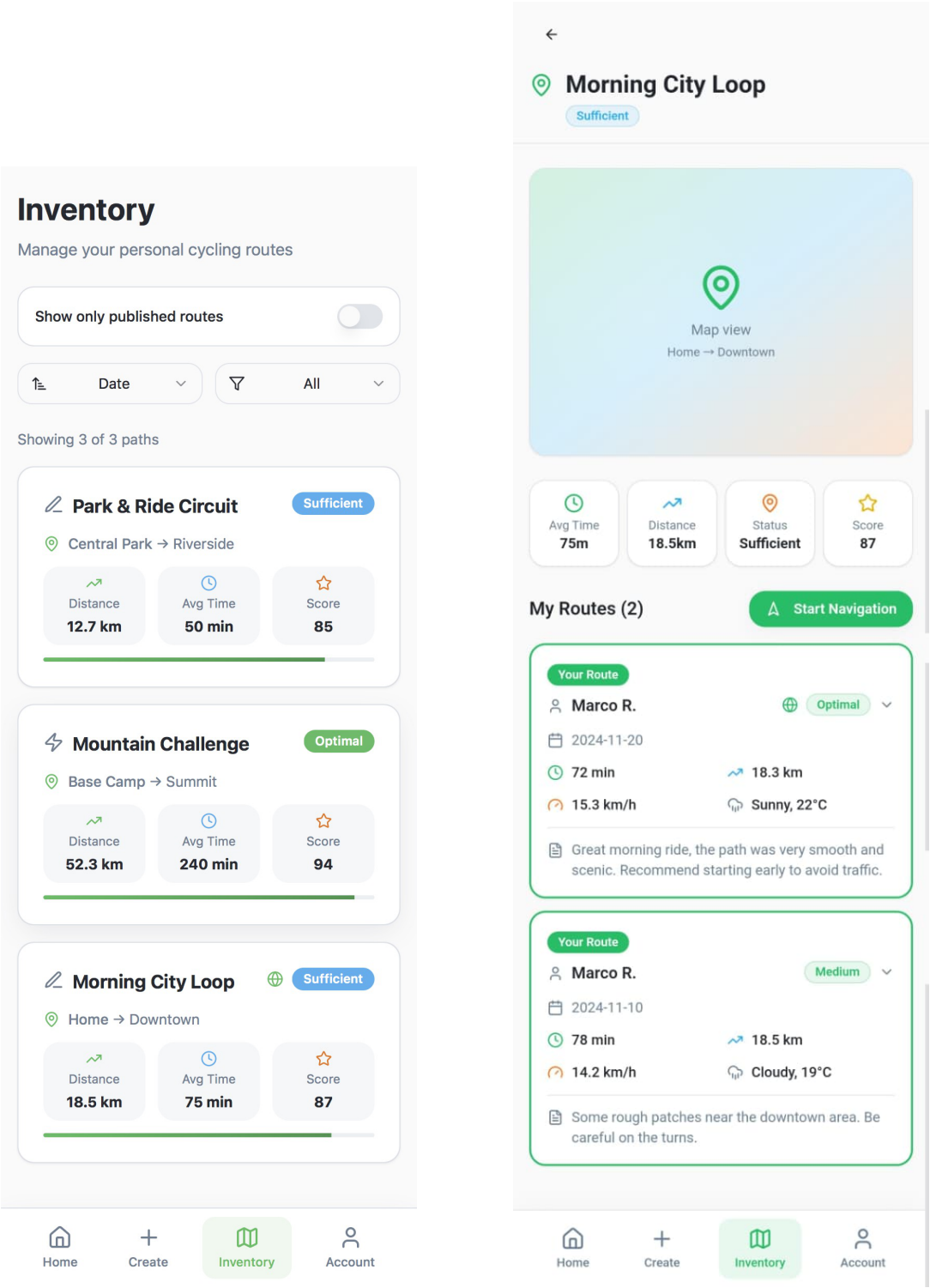


Figure 7: Screens of *Inventory* and *Path's details*

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

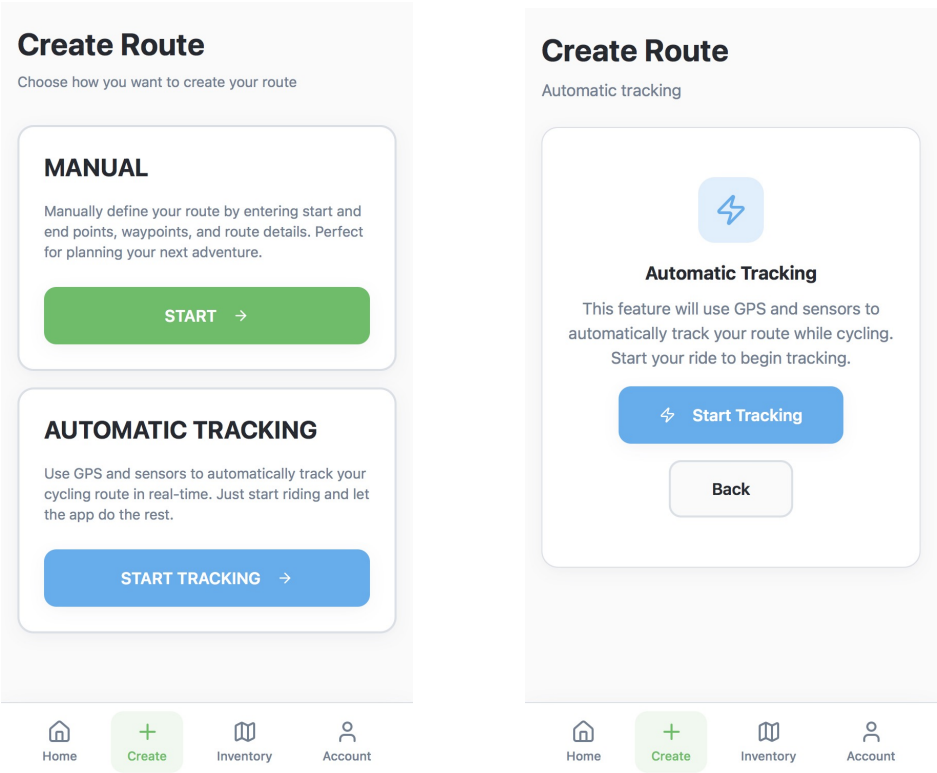


Figure 8: Screens of *Create Route* and *Create Route (Automatic Mode)*

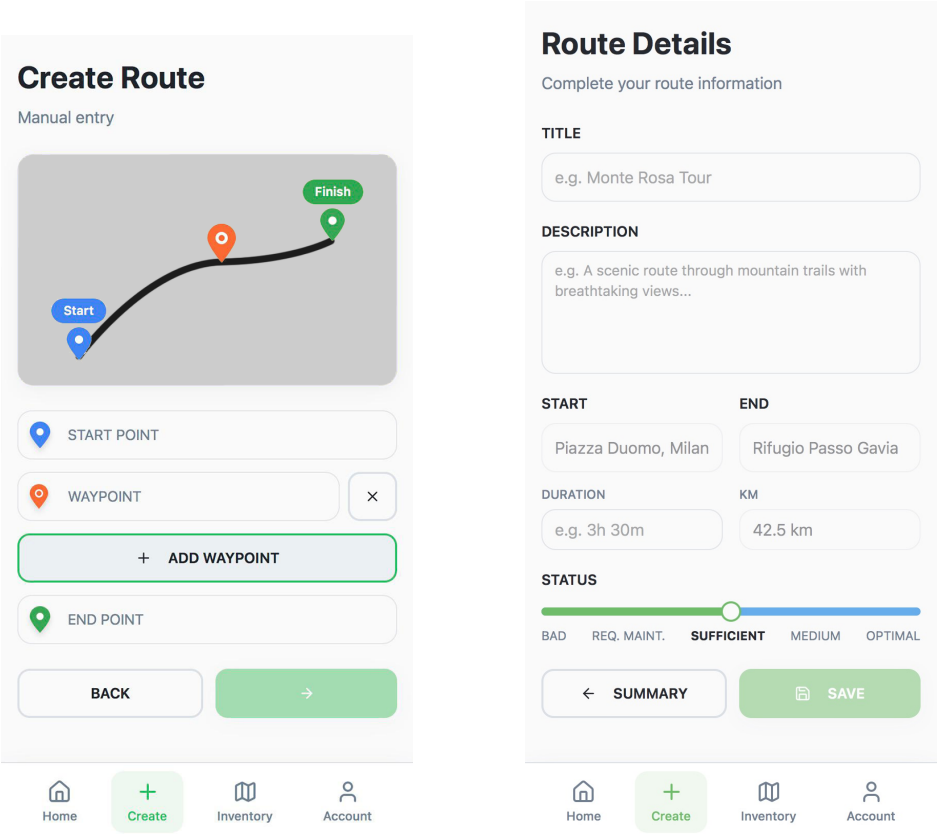


Figure 9: Screens of *Create Route (Manual Mode)* and *Route Details*

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.1.2 Hardware Interfaces

The platform is designed to be used on mobile devices taking advantage of their sensors.
The system interacts with several hardware components to provide its functionality, especially for the automatic path acquisition. In particular, the system uses the GPS sensor to track the cyclist's position in real time, while the accelerometer and gyroscope are used to detect and analyze the characteristics of the path.

3.1.3 Software Interfaces

The system implements a mapping solution based on pre-existing tools such as *OpenStreetMap* and its associated features. This allows users to create and visualize paths and to follow them using the navigation system.

3.1.4 Communication Interfaces

Cyclists use the Internet to access to the platform. An Internet connection is strictly required when users search for a path or publish a route as well as when the system indirectly communicates with the weather service to retrieve meteorological information to enrich a route before saving it.
To prevent data breaches, the software must protect exchanged data and avoid exposing sensitive information, such as the location of a user in real time or the credentials. To achieve this, communications must be secured using HTTPS to encrypt data during transmission.

3.2 Functional Requirements

Sign-up e Login

- **R1:** The platform allows the Cyclists to create an account.
- **R2:** The platform allows Cyclists to authenticate.
- **R3:** The platform allows the access as a Guest.
- **R4:** The platform allows Cyclists to view their personal account.
- **R5:** The platform allows Cyclists to modify their personal account.

Route Creation and Publication

- **R6:** The platform allows the Cyclists to create a new route through a manual creation process.
- **R7:** The platform allows the Cyclists to create a new route through an automatic creation process.
- **R8:** The platform allows the Cyclists to confirm and complete the automatically acquired information.
- **R9:** The platform collects data from the device sensors.
- **R10:** The platform collects weather data and enriches the path's statistics.
- **R11:** The platform allows the Cyclists to evaluate the status of a route.
- **R12:** The platform allows the Cyclists to publish/unpublish a route.
- **R13:** The platform allows the Cyclists to delete a route.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

Route Visualization

- **R14:** The platform allows the Cyclists to visualize a list of paths in the personal inventory with the possibility to filter them.
- **R15:** The platform allows the User to view the details of a path such as the map, the statistics and the status.
- **R16:** The platform allows the User to visualize the details of the routes related to a path.

Route Search and Navigation

- **R17:** The platform allows the User to search a path by setting the start and the end points with a kilometer tolerance.
- **R18:** The platform suggests to the User a list of available routes ranked by score.
- **R19:** The platform supports path navigation by allowing users to follow routes that have been previously created.

Scoring, Merging and Ranking

- **R20:** The platform merges the status when more than one is present.
- **R21:** The platform computes a score for each path based on path's status and average speed.

3.2.1 Use Case Diagram

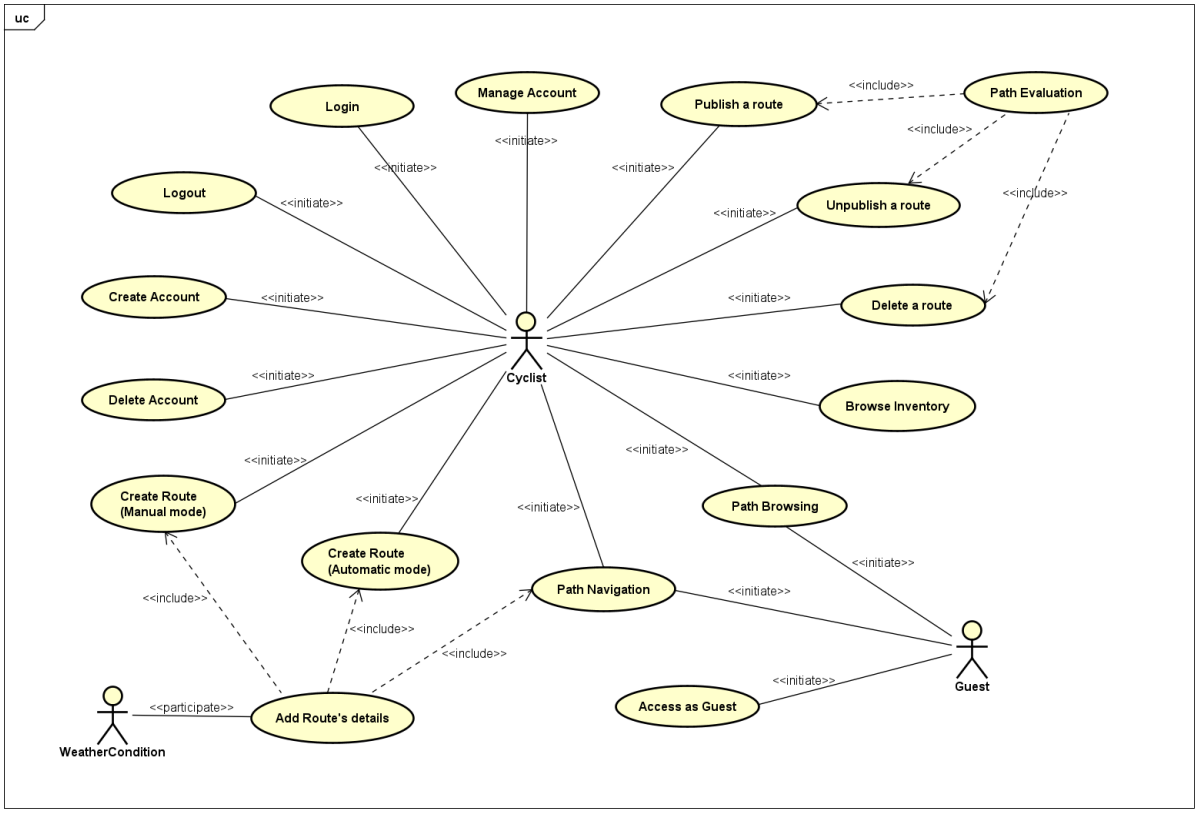


Figure 10: Use Case Diagram.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.2.2 Use Cases

- **UC1 - Create Account**

Name	Create Account
Actors	Cyclist
Entry Condition	The Cyclist has not an account yet and they are on the authentication page
Event Flow	1. The Cyclist presses the button *Sign Up* 2. The system shows a registration form 3. The Cyclist compiles the form providing his name, e-mail and password 4. The Cyclist presses the button *Sign Up* 5. The system checks if the email has not already been used
Exit Condition	The Cyclist has successfully registered to the platform and the system shows the Homepage
Exception	There already exist a cyclist with those e-mail; the system return to the entry condition showing an error message *Cyclist already registered*

- **UC2 - Login**

Name	Login
Actors	Cyclist
Entry Condition	The Cyclist has already created an account and he's on the authentication page
Event Flow	1. The Cyclist presses the *Login* button 2. The system shows the login form 3. The Cyclist compiles the form inserting e-mail and password 4. The Cyclist presses the *Login* button 5. The system checks if the credentials are correct
Exit Condition	The Cyclist has successfully logged to the platform and the system shows the Homepage
Exception	The credentials are not correct so the system shows an error message and shows again the [fig:LoginSignUp]authentication page

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC3 - Guest Access**

Name	Guest Access
Actors	Guest
Entry Condition	The guest is on the authentication page
Event Flow	1. The Guest presses the *Continue as Guest* button 2. The system shows an overlay with a message 'Continuing as a Guest, you can access only to limited features. Are you sure?' 3. The Guest presses the confirm button *Continue as Guest* on the overlay
Exit Condition	The system shows the Guest Homepage (the other screens are locked)
Exception	The Guest presses the *Cancel* button and the system closes the overlay

- **UC4 - Logout**

Name	Logout
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Account* button 2. The system shows the account page 3. The Cyclist presses the *Logout* button on the top of the page 4. The system shows an overlay with a message 'Do you want to logout?' 5. The Cyclist presses the confirm button *Logout* on the overlay
Exit Condition	The authentication page is shown
Exception	The Cyclist presses the *Cancel* button and the system closes the overlay

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC5 - Manage Account**

Name	Manage Account
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Account* button 2. The system shows the account page 3. The Cyclist presses the *Edit Account* button 4. The system shows a form where the Cyclist can edit his credentials, his account photo and his username 5. The Cyclist edit his information and presses the *Confirm* button
Exit Condition	The system shows the Account Page with updated information
Exception	The Cyclist inserts invalid data and the system shows an error message without updating the account

- **UC6 - Delete Account**

Name	Delete Account
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Account button* 2. The system shows the Account page 3. The Cyclist presses the *Delete Account* button on the bottom of the page 4. The system shows an overlay with a message "Are you sure do you want to delete your account?" 5. The Cyclist presses the confirm button *Delete Account* on the overlay
Exit Condition	The Cyclist is logged out from the platform, the system shows the authentication page and delete all the information related to the Cyclist
Exception	The Cyclist presses the *Cancel* button and the system closes the overlay

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC7 - Create Route (Manual mode)**

Name	Create Route (Manual mode)
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Create* button 2. The system shows a page where the Cyclist has to choose the creation mode of the route 3. The Cyclist presses on *Start* button to manually create a route 4. The system shows the manual creation page composed by a dynamic map that connects the Start Point, the Way Points and the End Point inserted by the Cyclist 5. The Cyclist fills all the geographical point 6. The Cyclist presses the → button to continue
Exit Condition	The system shows the page to add route's details and executes **UC9 - Add Route's details**
Exception	The Cyclist presses *Back* button to go back and change the creation mode

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC8 - Create Route (Automatic mode)**

Name	Create Route (Automatic mode)
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Create* button 2. The system shows a page where the Cyclist has to choose the creation mode of the route 3. The Cyclist presses on *Start tracking* button to start the automatically recording 4. The system shows the automatic creation page composed with a button to effectively start the tracking when the Cyclist is ready 5. The Cyclist presses the *Start tracking* button to start the recording 6. The system uses the device's sensors such as GPS, accelerometer and gyroscope to detect all the movements of the cyclist in order to obtain the set of streets and its status 7. The Cyclist presses the *Stop tracking* button to conclude the recording 8. The system shows a page with a map to check the set of streets allowing the Cyclist to edit and confirm 9. The Cyclist revises any errors and presses the confirm button
Exit Condition	The system shows the page to add route's details and executes **UC9 - Add Route's details**
Exception	The Cyclist presses *Back* button to go back and change the creation mode

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC9 - Add Route's details**

Name	Add Route's details
Actors	Cyclist, WeatherCondition
Entry Condition	The system shows the page to add route's details with precompiled and editable forms such as *Start Point*, *End Point*, *Duration* and *Distance* and other parameters to be filled
Event Flow	1. The Cyclist provides missing information regarding the details of the route, adding the *Description* through the forms and eventually edits the other precompiled forms 2. The Cyclist evaluates the status of the route through the slider (deciding among bad, require maintenance, sufficient, medium, optimal) 3. The Cyclist presses the *Save* button 4. The system asks for the weather condition in the date of the route 5. The WeatherCondition provides these information 6. The system enriches the route's details with data provided by WeatherCondition 7. The system checks for the existence of another path with a set of streets compatible with the set of streets collected during route creation 8. If it does not exist, the system creates a new path for the current route 9. The system adds the new route to the related path 10. The system updates the account statistics, such as number of routes, total distance and total time.
Exit Condition	The system saves the route into the related path and shows the path's details page
Exception	

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC10 - Browse inventory**

Name	Browse inventory
Actors	Cyclist
Entry Condition	The Cyclist is logged into the system
Event Flow	1. The Cyclist presses the *Inventory* button 2. The system shows the Cyclist's inventory with the options to filter the paths displayed by status or by public paths and the options to sort the paths by date, distance and score 3. The Cyclist presses on the path he wants to visualize
Exit Condition	The system shows the path's details page where the user can visualizes the map, the statistics (such as average time, distance, status and score), the list of his routes related to that path and a button to start navigation
Exception	

- **UC11 - Path Browsing**

Name	Path Browsing
Actors	Cyclist, Guest
Entry Condition	The User is on the Homepage where he can visualize the parameters to perform the search and a list of top rated paths
Event Flow	1. The User inserts a *Start Point* and an *End Point* in the dedicated forms, chooses a *Tolerance* through the slider and then presses the *Search Paths* button 2. The system searches the paths that best fits the criteria and shows them ordered by *Score* 3. The User presses a path to select it
Exit Condition	The system shows the p path's details page
Exception	User go back and changes the selected path

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC12 - Path Navigation**

Name	Path Navigation
Actors	Cyclist, Guest
Entry Condition	The User is on the path's details page
Event Flow	1. The User presses the *Start Navigation* button 2. The system shows the navigation page with a map to guide the User along the path in real time 3. As soon as the User reaches the end point, the system stops the navigation and shows a message "Path completed" 4. The User presses the *Continue* button
Exit Condition	The User is a Cyclist and the system shows the page to add route's details and executes **UC9-Add Route's details**
Exception	The User is a Guest and the system shows a message "You have to login to save a route" and a button to perform authentication

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC13 - Publish a route**

Name	Publish a route
Actors	Cyclist
Entry Condition	Cyclist is in the inventory and the route is private
Event Flow	1. The Cyclist expands the view of the route that he wants to publish 2. The system shows the button to publish or delete the route 3. The Cyclist presses the *Publish* button 4. The system shows an overlay with the message *"All the information about this route will be visible to other users. Do you want to continue?"* 5. The Cyclist presses the *Continue* button on the overlay 6. The system executes **UC15 - Path Evaluation** considering the route to be published and eventually also the others already published 7. If the path is not already public, the system makes it public 8. The system makes the route public and now it is visible by other users inside the related path. 9. The system updates the number of published routes in the account statistics.
Exit Condition	The system shows the *Unpublish* button
Exception	

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC14 - Unpublish a route**

Name	Unpublish a route
Actors	Cyclist
Entry Condition	Cyclist is in the inventory and the route is public
Event Flow	1. The Cyclist expands the view of the route that he wants to unpublish 2. The system shows the button to unpublish or delete the route 3. The Cyclist presses the *Unpublish* button 4. The system shows an overlay with the message *"All the information about this route will be no more visible to other users. Do you want to continue?"* 5. The Cyclist presses the *Continue* button on the overlay 6. The system checks the existence of at least another public route for that path 7. If there is, the system executes **UC15 - Path Evaluation**, excluding the route to be unpublished 8. Else the system makes the path not public until another route will be published again 9. The system makes the route private and now it is visible only in the inventory 10. The system updates the number of published routes in the account statistics.
Exit Condition	The system shows the *Publish* button
Exception	

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC15 - Path Evaluation**

Name	Path Evaluation
Actors	Cyclist
Entry Condition	The system has to update the status and the score of a path
Event Flow	1. The system gets all the public routes having the most recent date available in the path to be updated 2. If there is only one route, the system updates the path status with the status of that route 3. Else, the system updates the path status with the value corresponding to the status shared by the largest number of routes (in case of a tie, the system computes the average by treating the status values as numerical values from 1 to 5) 4. After the status computation, the system updates the other path's statistics, such as the average time and distance 5. The system calculates and updates the path's score based on path's status and average speed
Exit Condition	The system has updated the status and the score of a path
Exception	

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **UC16 - Delete a route**

Name	Delete a route
Actors	Cyclist
Entry Condition	Cyclist is in the inventory
Event Flow	1. The Cyclist expands the view of the route that he wants to delete 2. The system shows the button to publish/unpublish or delete the route 3. The Cyclist presses the *Delete* button 4. The system shows an overlay with the message *"All the information about this route will be deleted forever. Do you want to continue?"* 5. The Cyclist presses the *Continue* button on the overlay 6. The system retrieves the other private and public routes of the path related to the current route 7. If the route is the last one of that path, the system deletes the path 8. Else if the route is published 8.1. If there are other published routes, the system executes **UC15 - Path Evaluation** 8.2. Else the system makes the path private 9. The system deletes the route 10. The system updates the account statistics, such as number of routes, number of published routes, total distance and total time.
Exit Condition	The system shows the updated inventory page
Exception	

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.2.3 Sequence Diagrams

- **SD1 - Create Account**

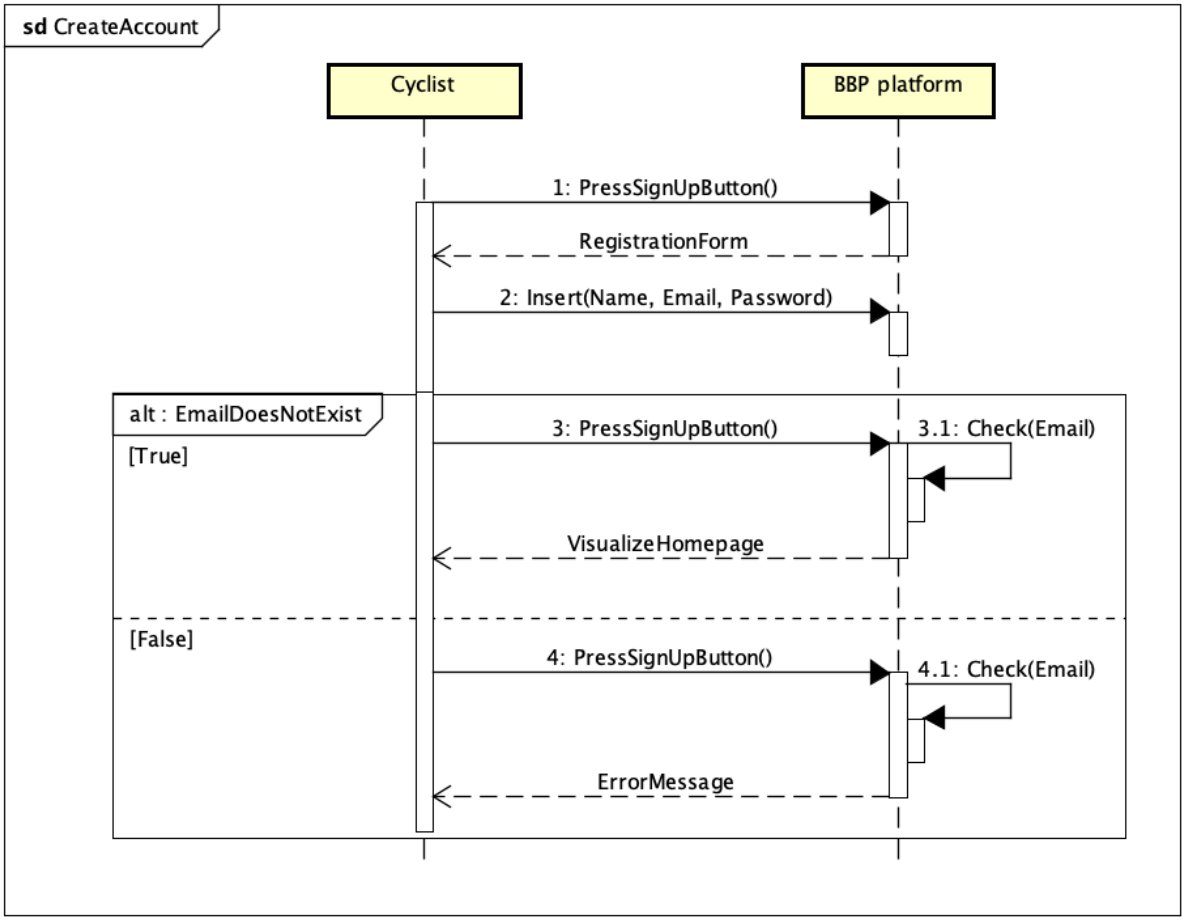


Figure 11: Sequence Diagram related to UC1 - Create Account

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD2 - Login**

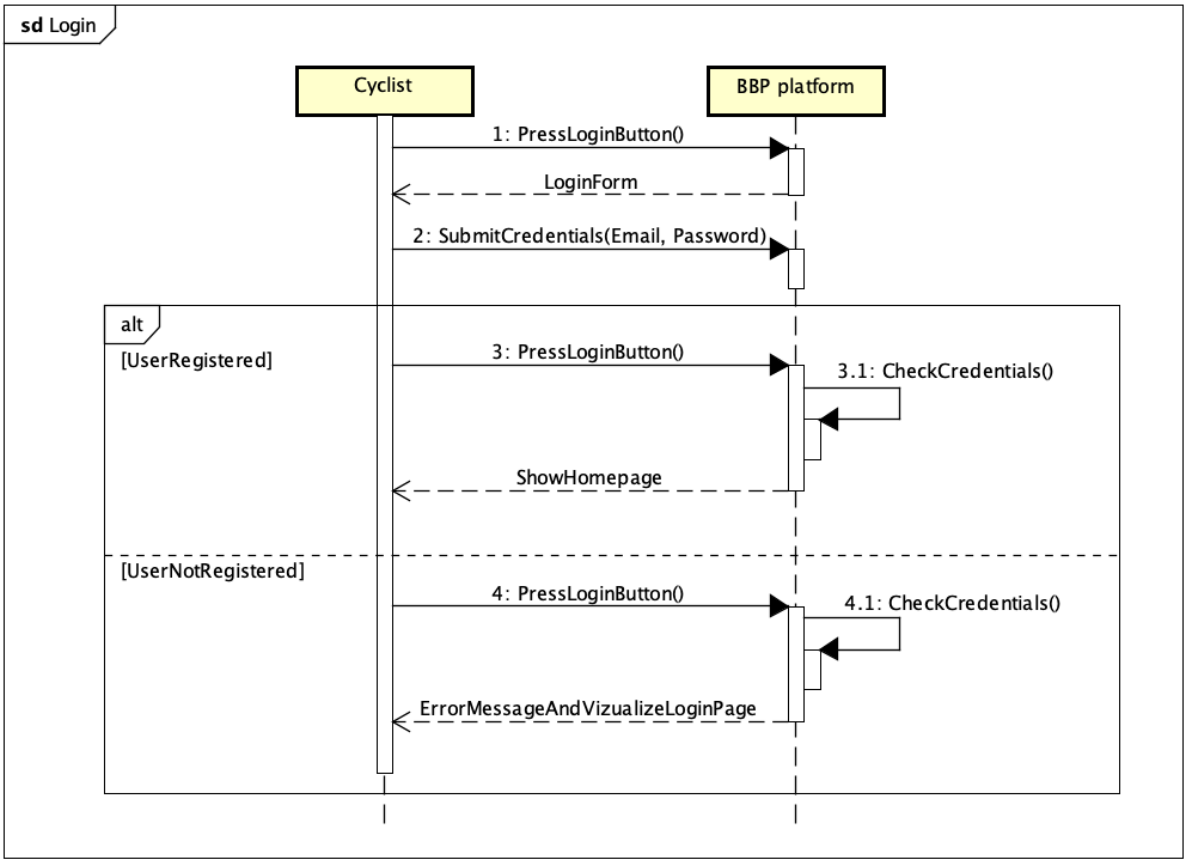


Figure 12: Sequence Diagram related to UC2 - Login

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD3 - Guest Access**

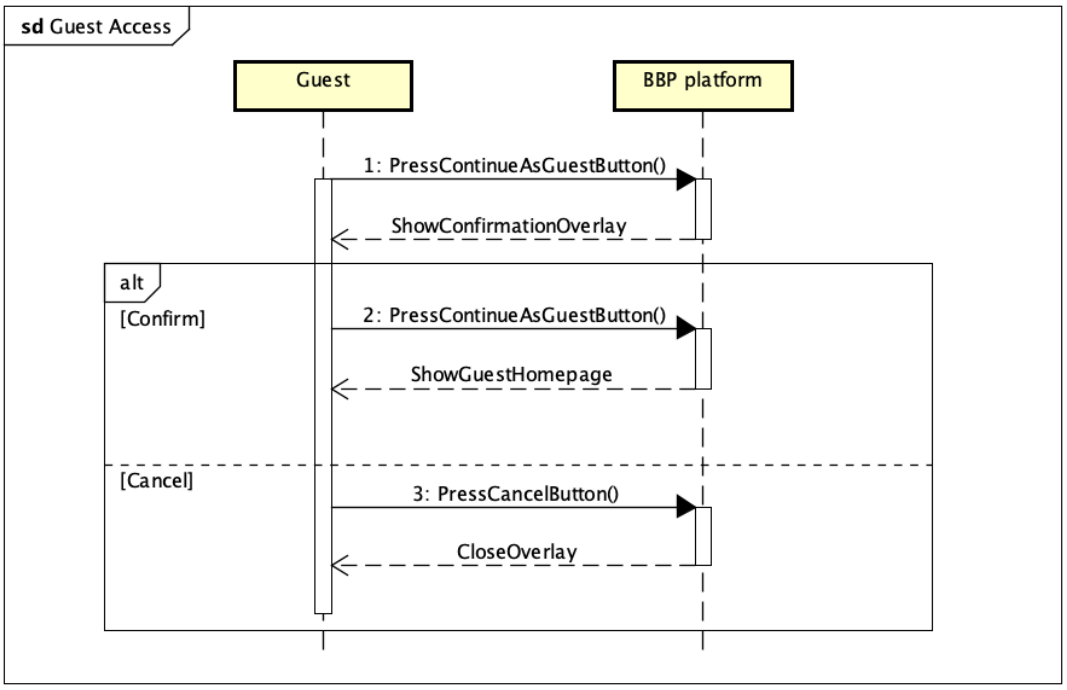


Figure 13: Sequence Diagram related to UC3 - Guest Access

- **SD4 - Logout**

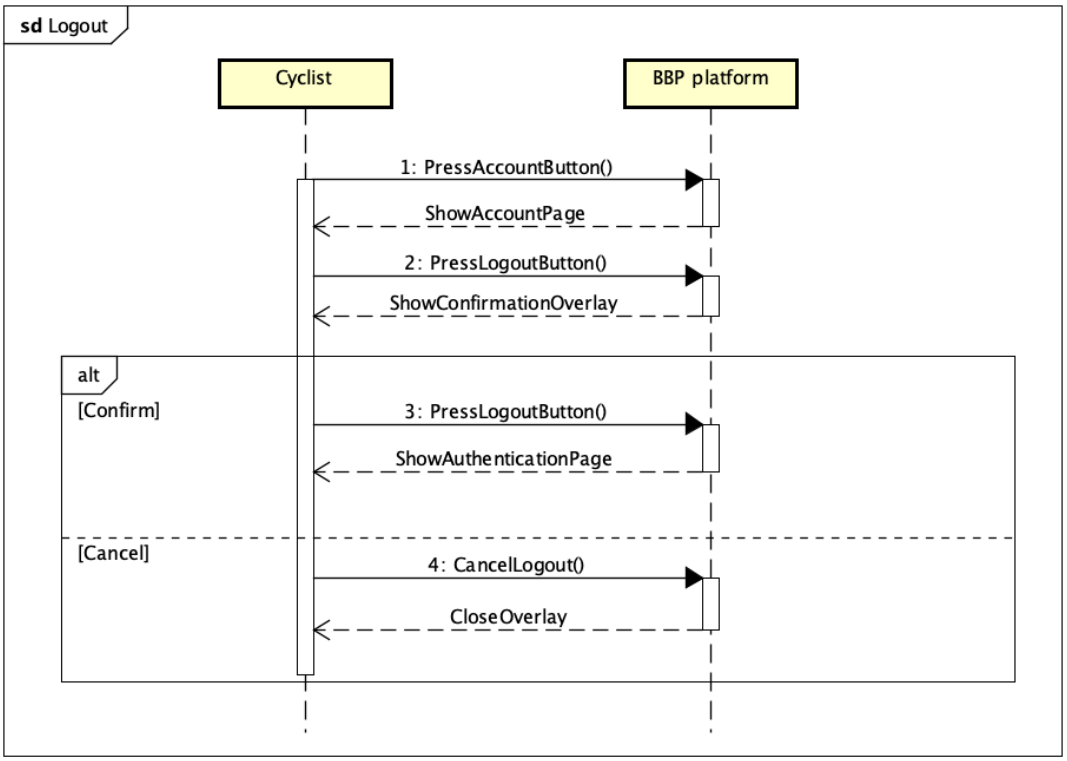


Figure 14: Sequence Diagram related to UC4 - Logout

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD5 - Manage Account**

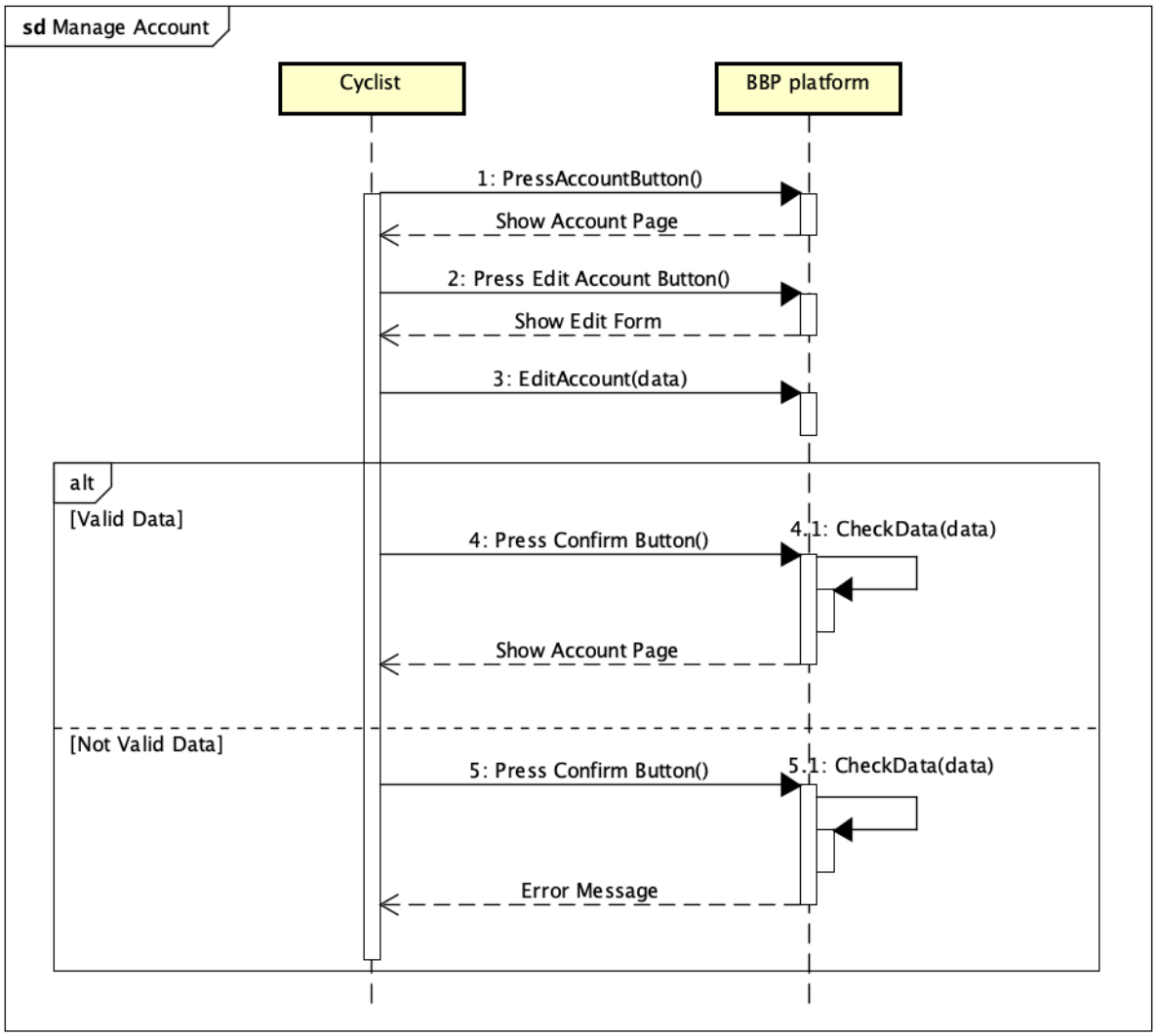


Figure 15: Sequence Diagram related to UC5 - Manage Account

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD6 - Delete Account**

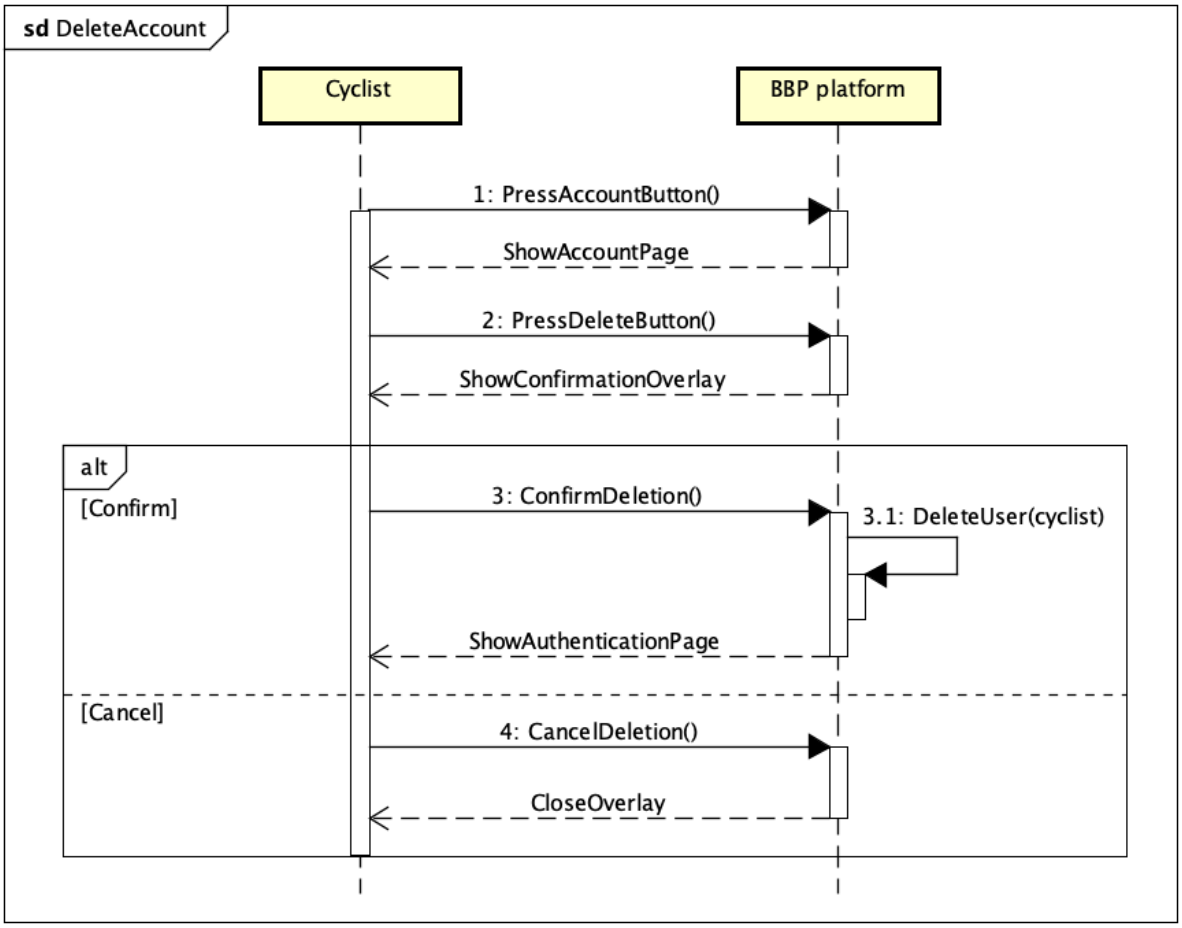


Figure 16: Sequence Diagram related to UC6 - Delete Account

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD7 - Create Route (Manual Mode)**

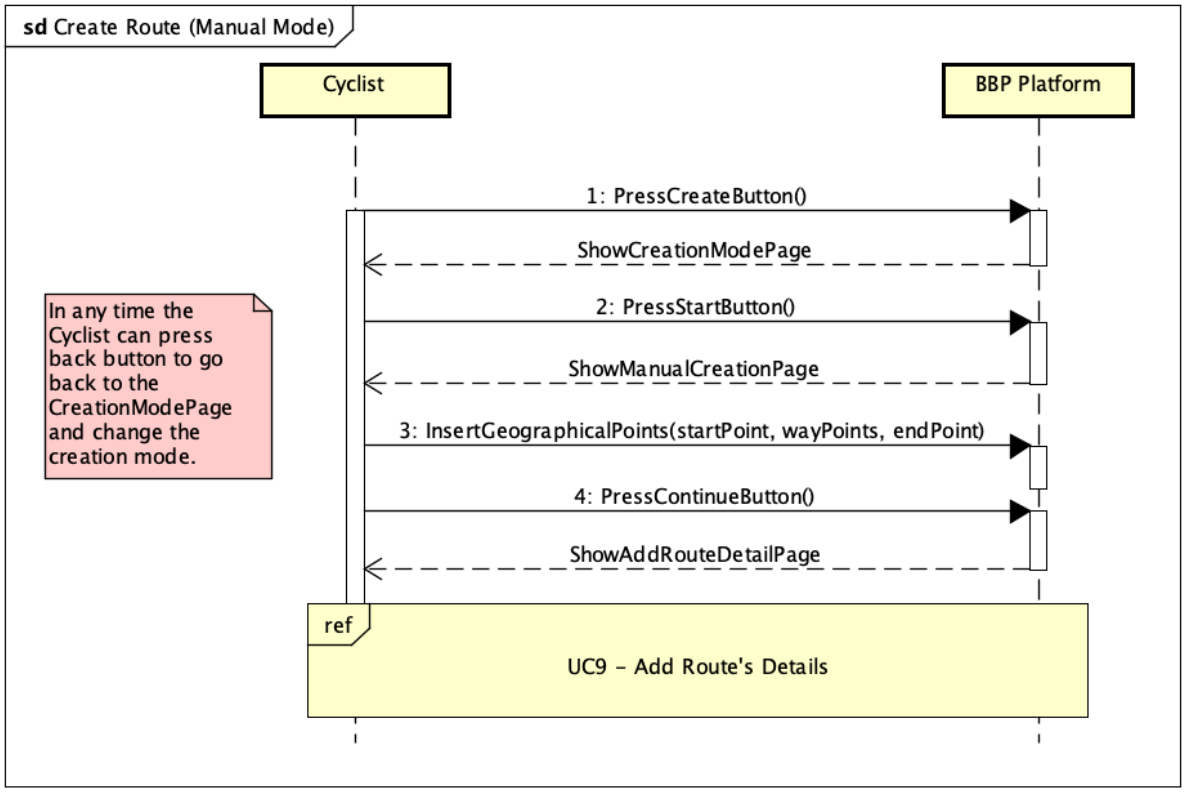


Figure 17: Sequence Diagram related to UC7 - Create Route (Manual Mode)

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD8 - Create Route (Automatic Mode)**

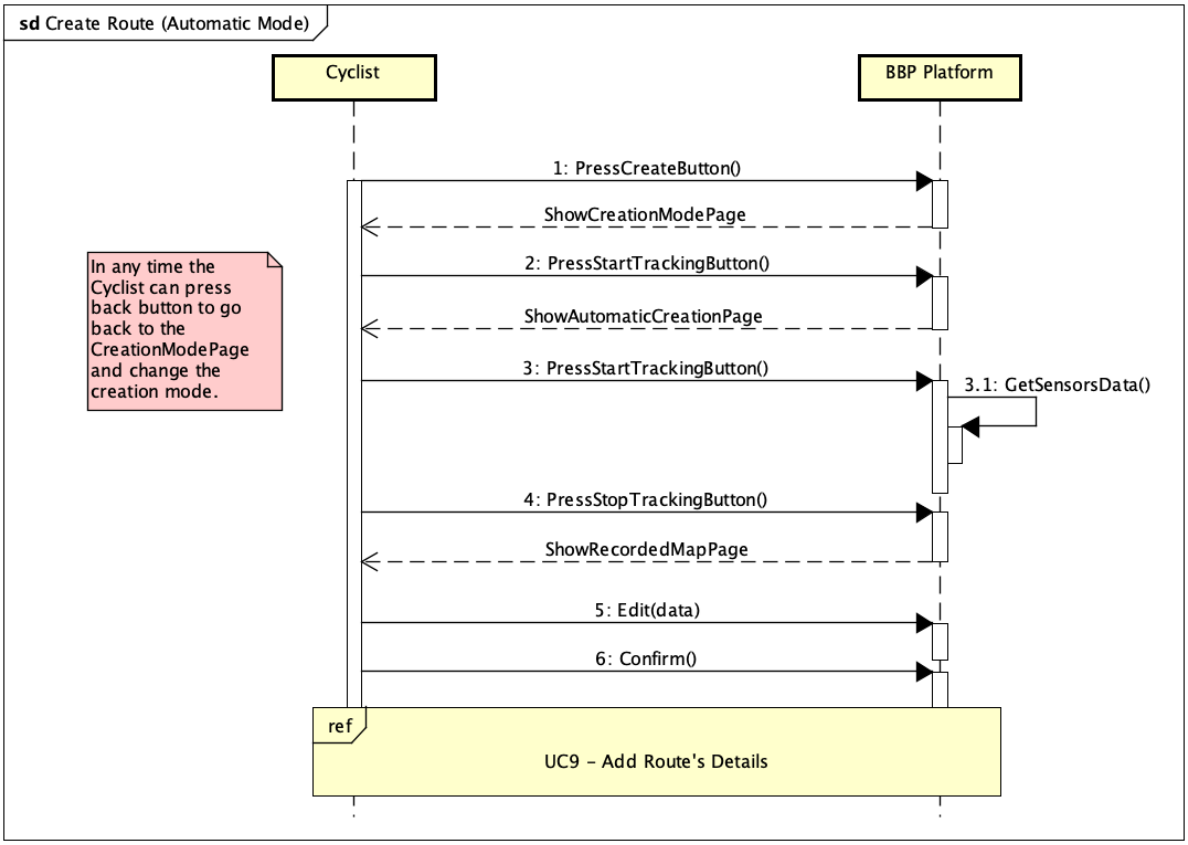


Figure 18: Sequence Diagram related to UC8 - Create Route (Automatic Mode)

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD9 - Add Route's Details**

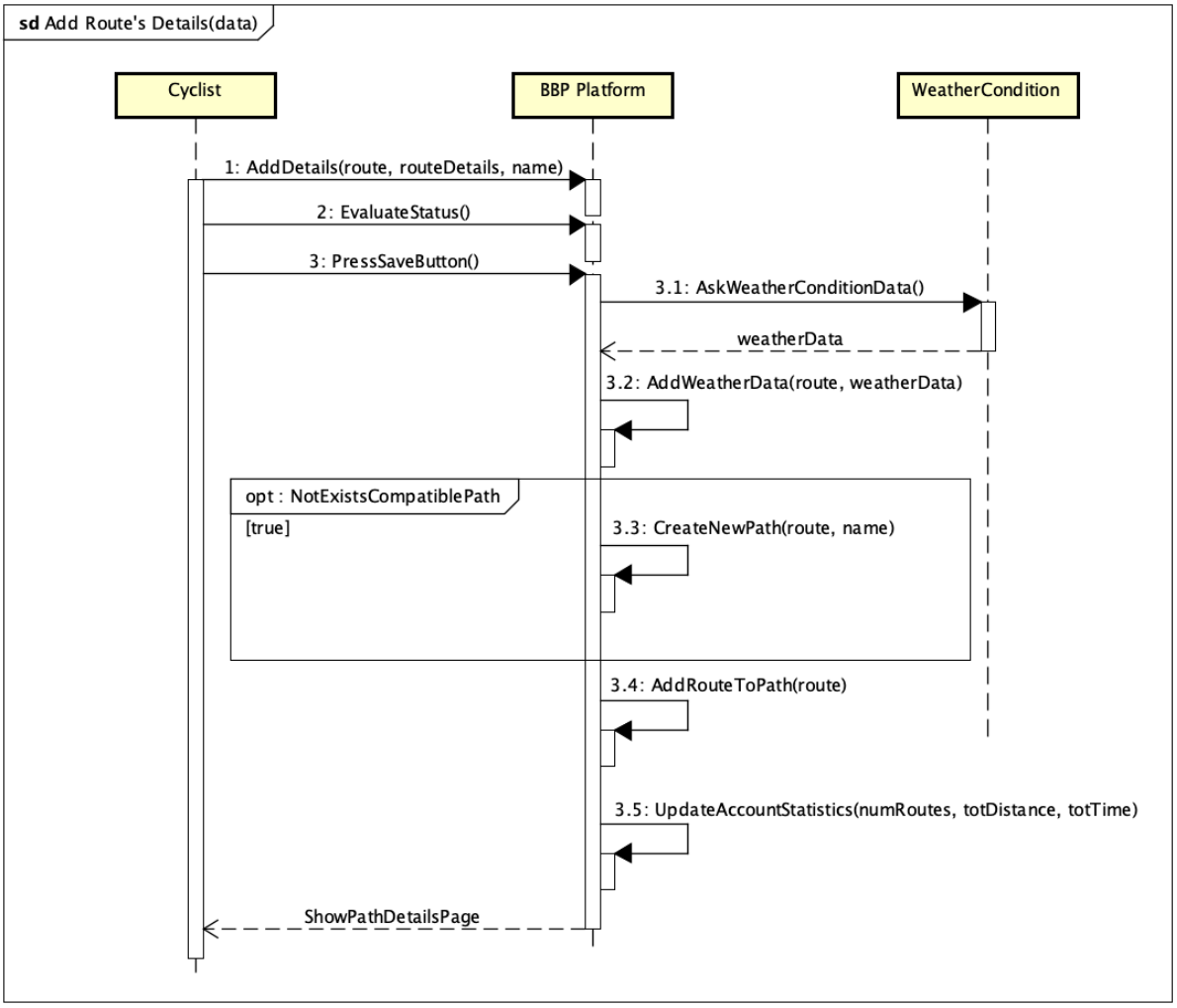


Figure 19: Sequence Diagram related to UC9 - Add Route's Details

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD10 - Browse Inventory**

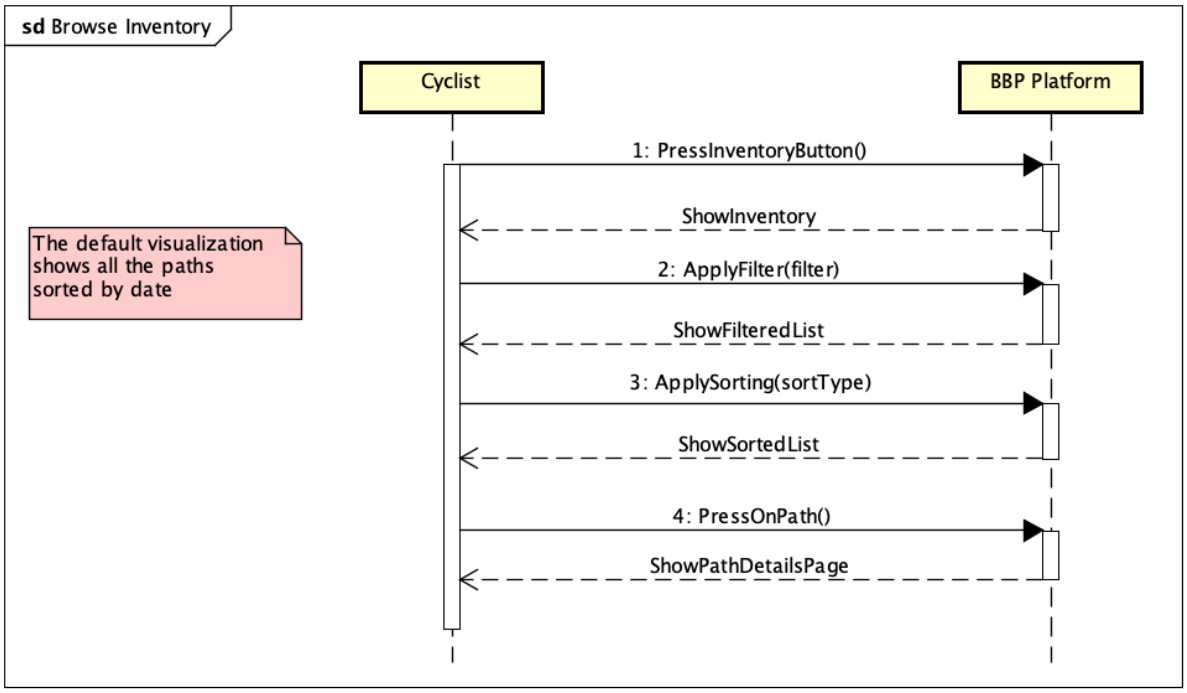


Figure 20: Sequence Diagram related to UC10 - Browse Inventory

- **SD11 - Path Browsing**

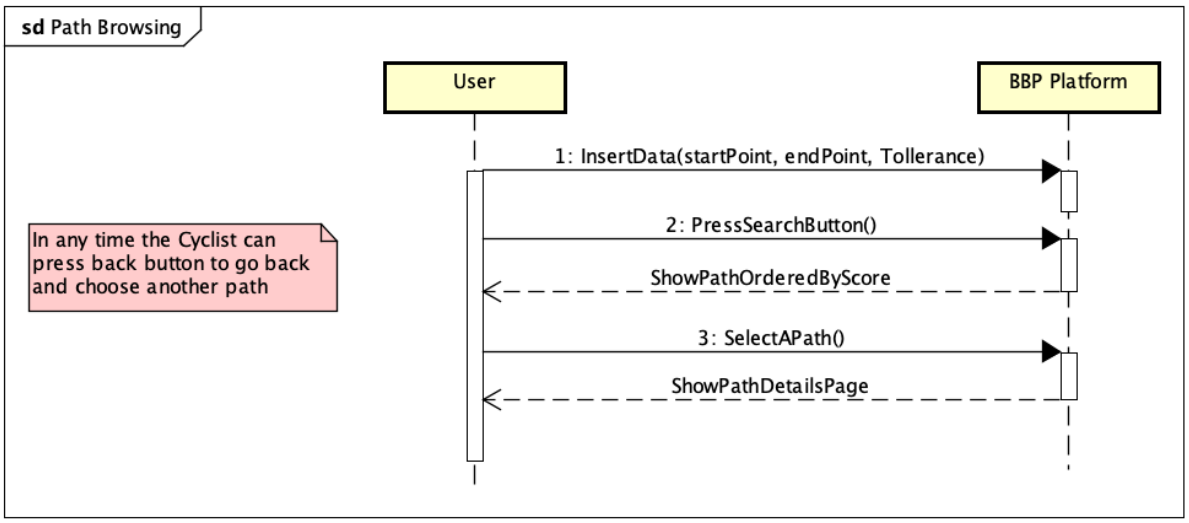


Figure 21: Sequence Diagram related to UC11 - Path Browsing

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD12 - Path Navigation**

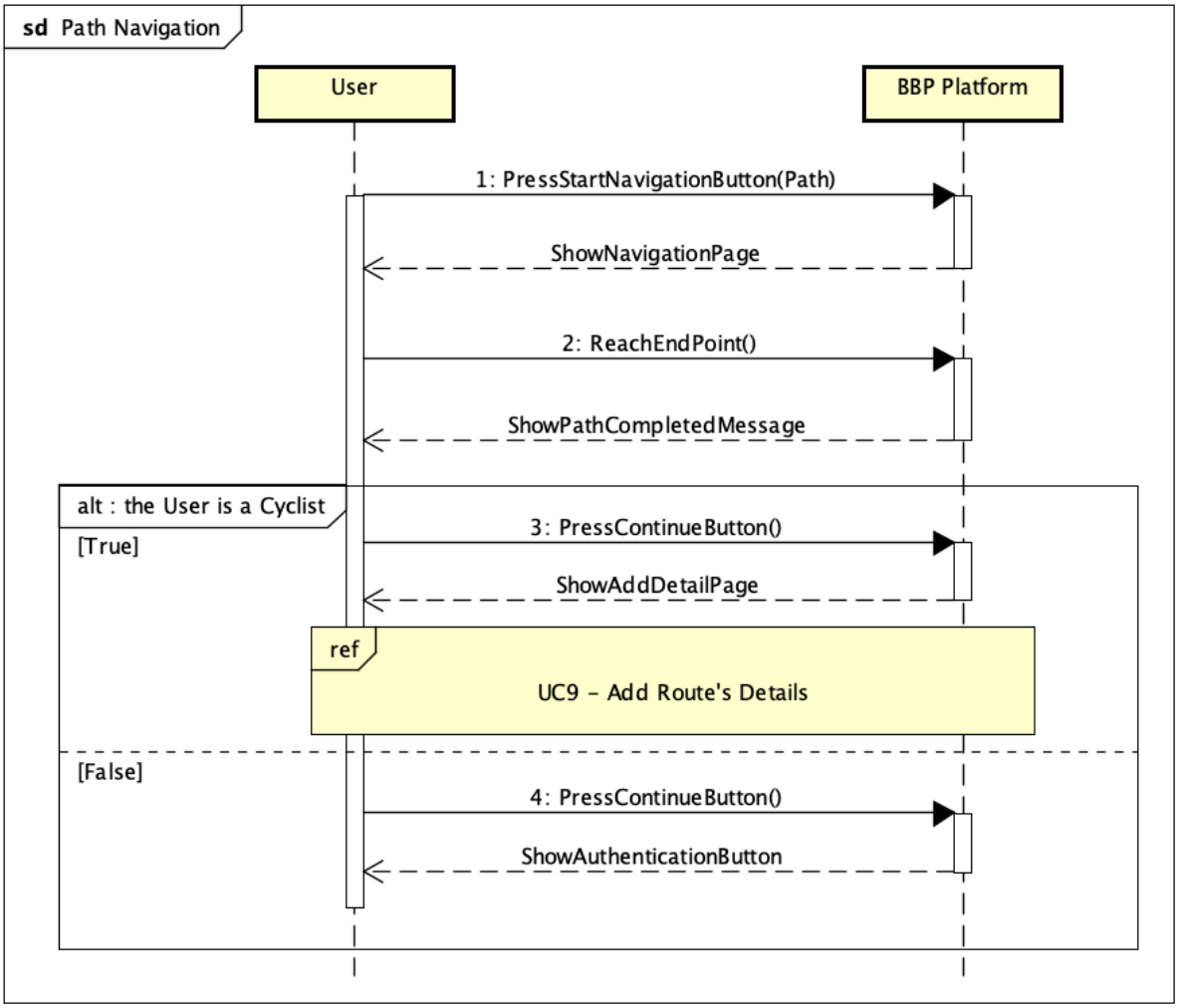


Figure 22: Sequence Diagram related to UC12 - Path Navigation

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD13 - Publish a route**

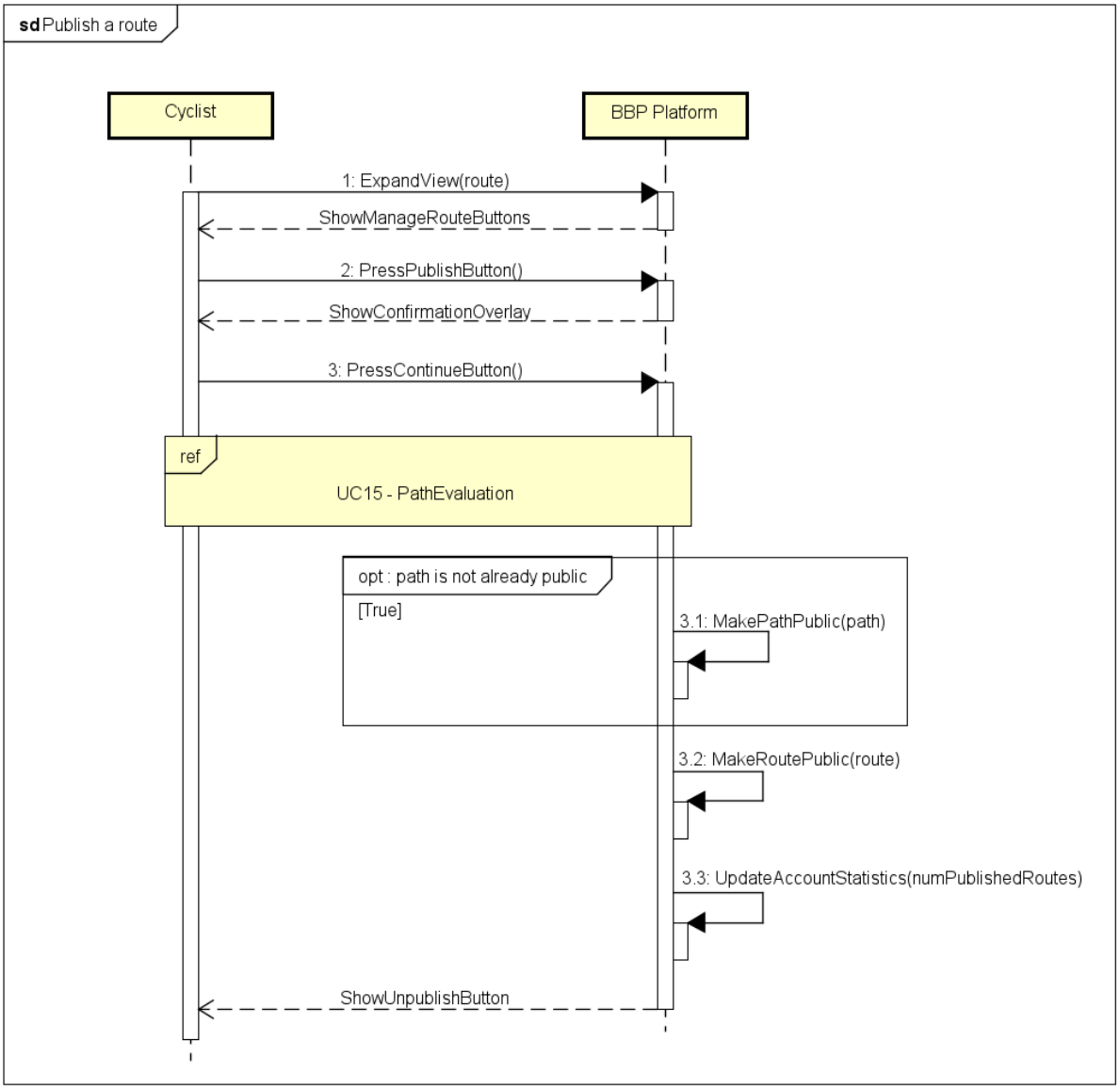


Figure 23: Sequence Diagram related to UC13 - Publish a route

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD14 - Unpublish a route**

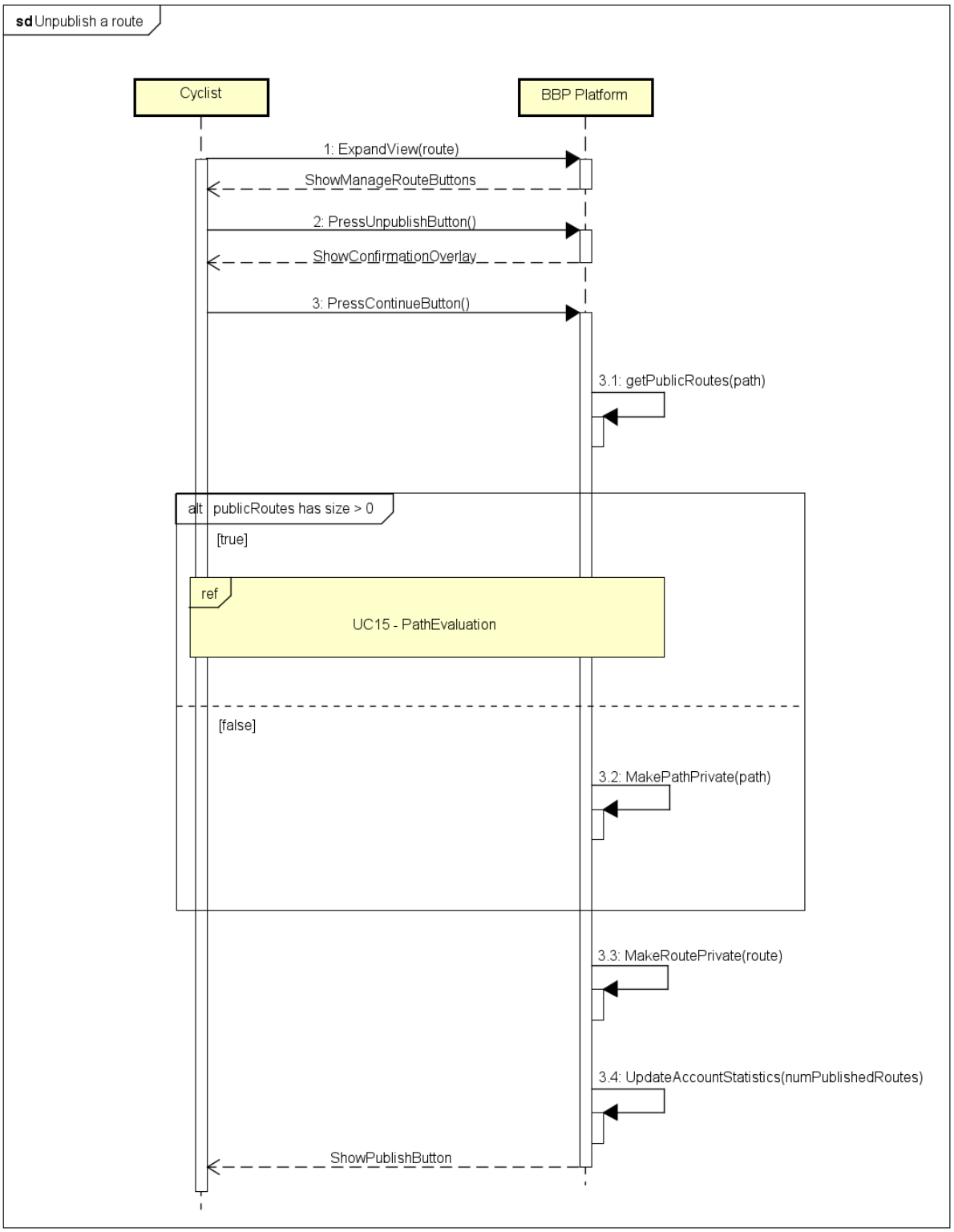


Figure 24: Sequence Diagram related to UC14 - Unpublish a route

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD15 - Path Evaluation**

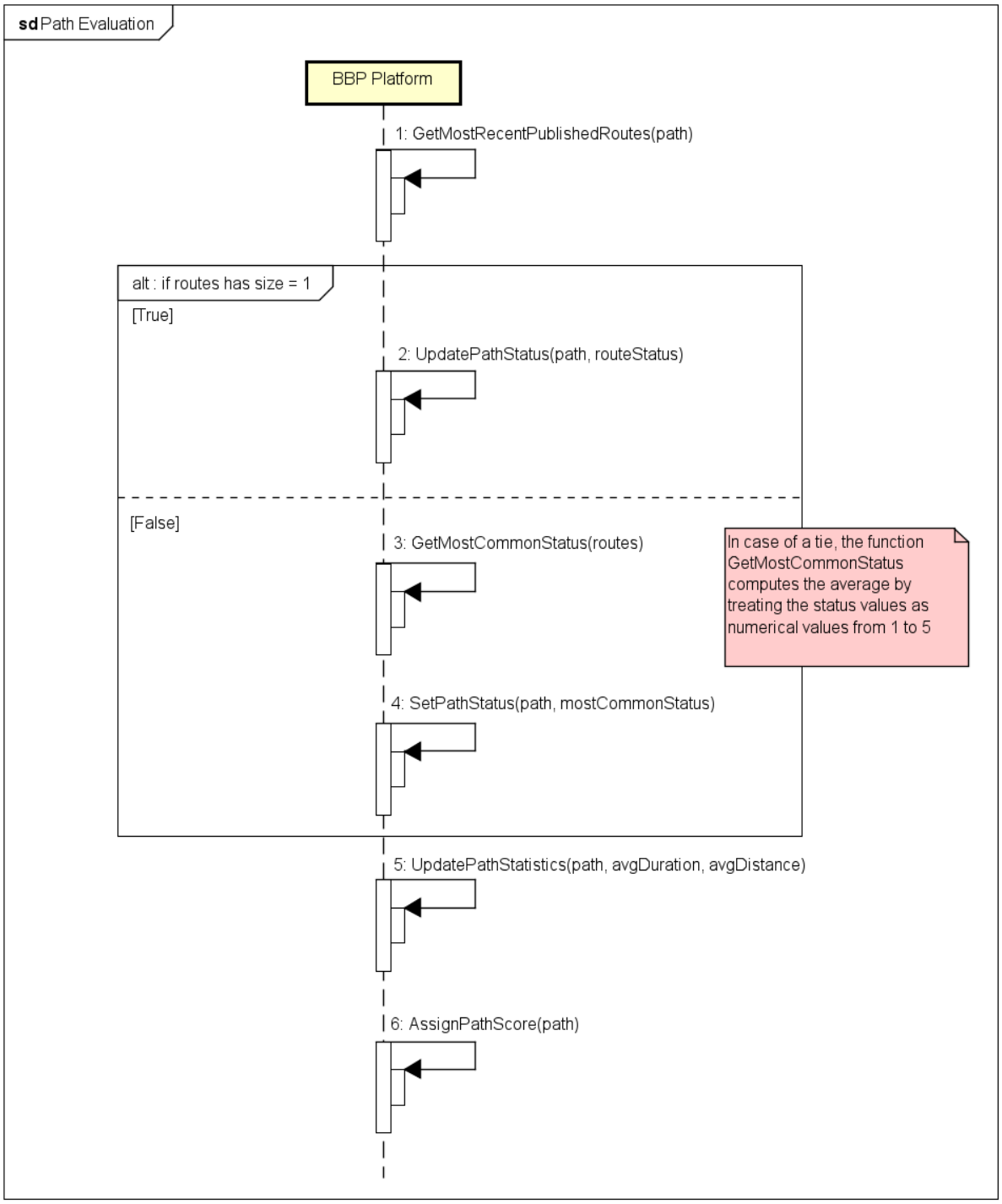


Figure 25: Sequence Diagram related to UC15 - Path Evaluation

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

- **SD16 - Delete a route**

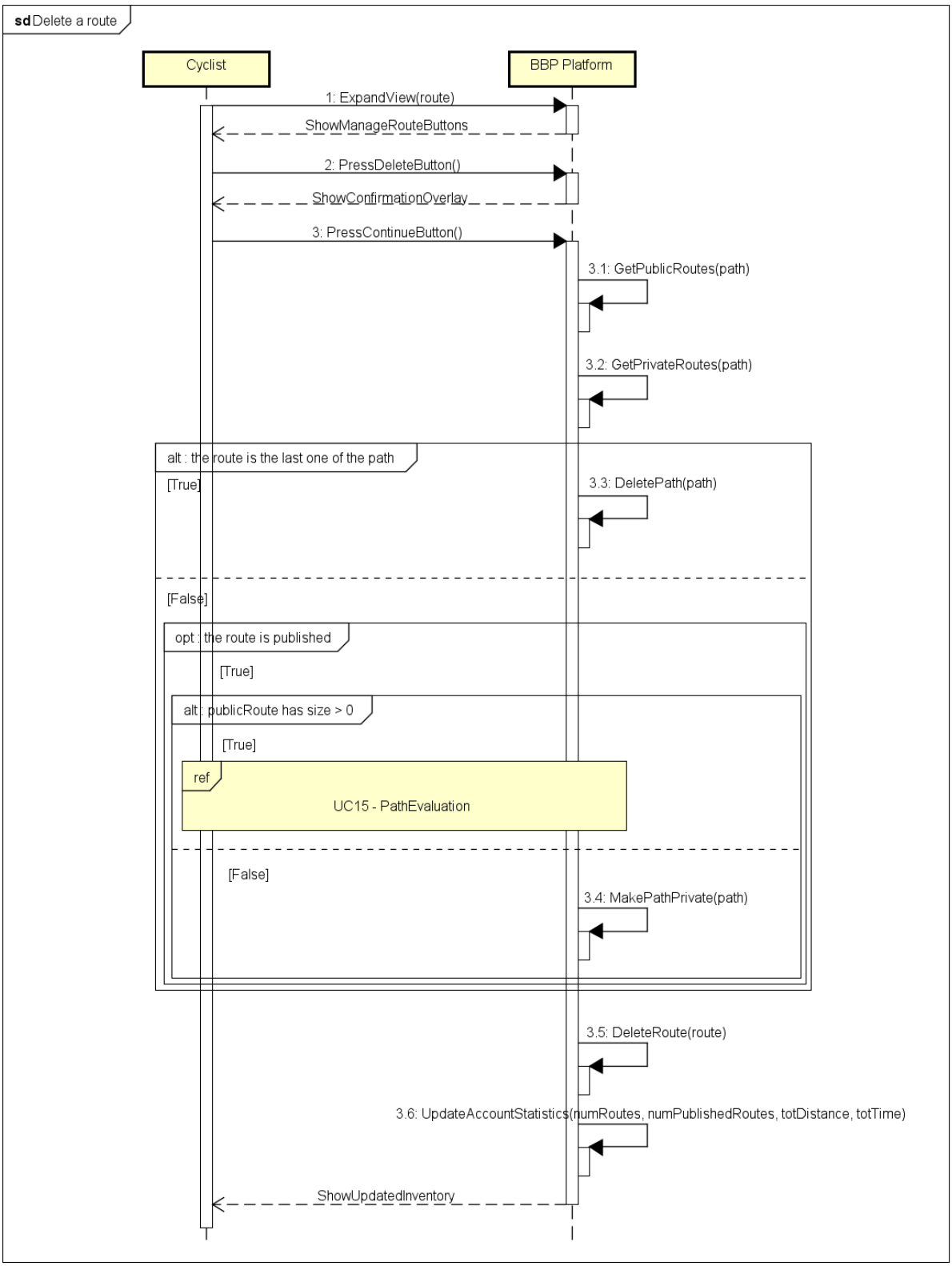


Figure 26: Sequence Diagram related to UC16 - Delete a route

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.2.4 Activity Diagram

- **AD16 - Delete a route**

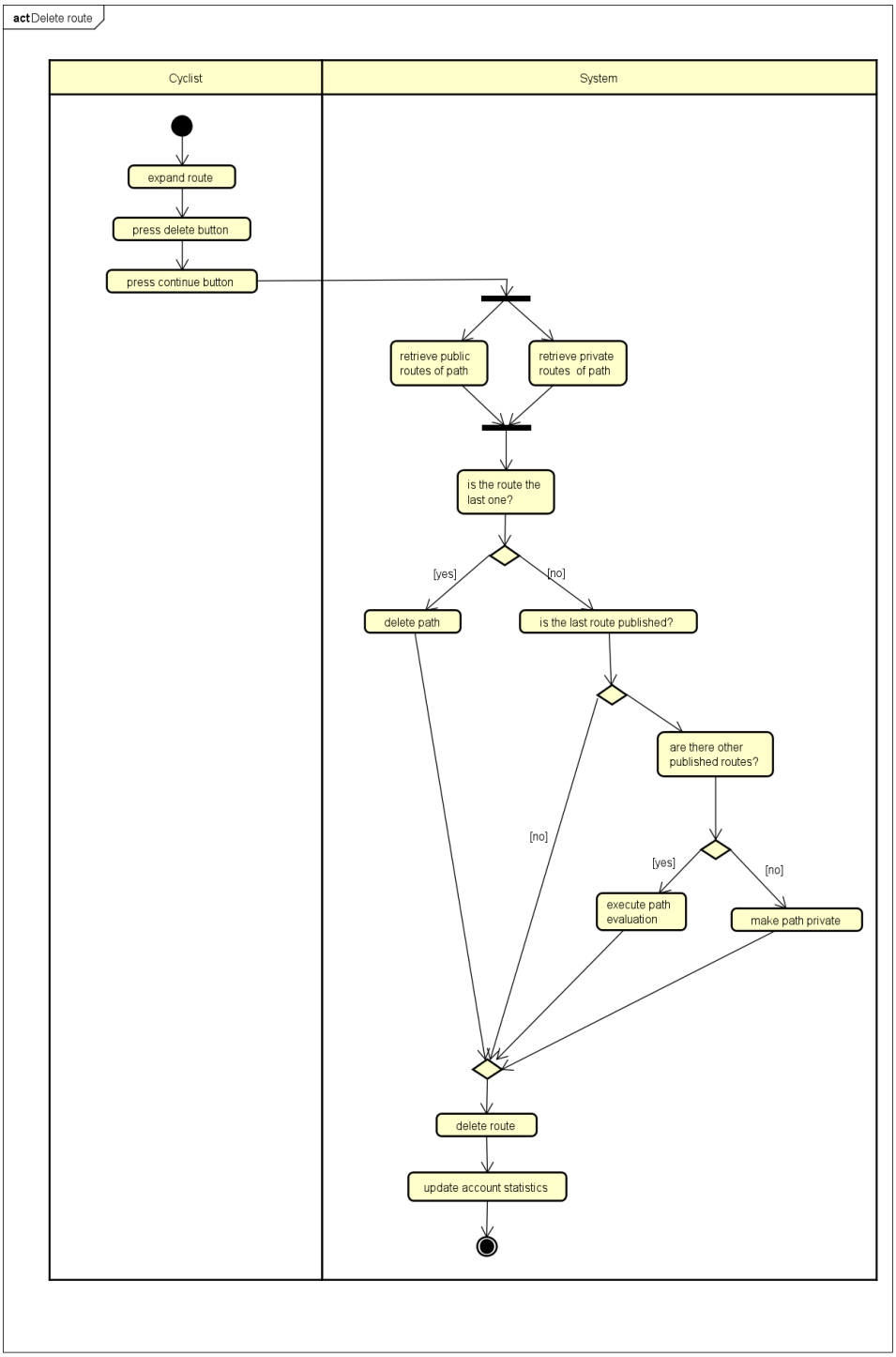


Figure 27: Activity Diagram related to UC16 - Delete a route

The activity diagram in the figure is intended to clarify the flow of actions of the use case **UC16 - Delete a route**. Since the operation is complex, two complementary representations are provided to allow its analysis from different perspectives.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.2.5 Requirement Mapping

G1: Cyclists can manually create bike routes or alternatively be assisted by the system to automatically track the paths thanks to the mobile device's sensors. Subsequently, the paths can be reviewed and enriched with general information and with a status before being saved inside the inventory.	
R1: The platform allows the Cyclists to create an account. **R2:** The platform allows Cyclists to authenticate. **R4:** The platform allows Cyclists to view their personal account. **R5:** The platform allows Cyclists to modify their personal account. **R6:** The platform allows the Cyclists to create a new route through a manual creation process. **R7:** The platform allows the Cyclists to create a new route through an automatic creation process. **R8:** The platform allows the Cyclists to confirm and complete the automatically acquired information. **R9:** The platform collects data from the device sensors. **R10:** The platform collects weather data and enriches the path's statistics. **R11:** The platform allows the Cyclists to evaluate the status of a route. **R15:** The platform allows the User to view the details of a path such as the map, the statistics and the status. **R16:** The platform allows the User to visualize the details of the routes related to a path.	**D1:** The cyclist's account data is correct. **D2:** The cyclist remembers his credentials. **D3:** The cyclist manually adds a path only after actually completing it. **D4:** When creating and evaluating a route, the cyclist inputs accurate and realistic information. **D5:** Reliable meteorological information. **D6:** If cyclists use GPS, accelerometer and gyroscope we assume that they are accurate and work correctly **D9:** All users have a reliable internet connection.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

G2: Cyclists can visualize and manage their own inventory and search for their saved routes with their statistics, status and meteorological information provided by external services.	
R1: The platform allows the Cyclists to create an account. **R2:** The platform allows Cyclists to authenticate. **R4:** The platform allows Cyclists to view their personal account. **R5:** The platform allows Cyclists to modify their personal account. **R13:** The platform allows the Cyclists to delete a route. **R14:** The platform allows the Cyclists to visualize a list of paths in the personal inventory with the possibility to filter them. **R15:** The platform allows the User to view the details of a path such as the map, the statistics and the status. **R16:** The platform allows the User to visualize the details of the routes related to a path.	**D1:** The cyclist's account data is correct. **D2:** The cyclist remembers his credentials. **D5:** Reliable meteorological information. **D9:** All users have a reliable internet connection.

G3: Cyclists can publish their routes, sharing them with the community, creating a social component in the system where users can discover different paths proposed by others.	
R1: The platform allows the Cyclists to create an account. **R2:** The platform allows Cyclists to authenticate. **R3:** The platform allows the access as a Guest. **R4:**The platform allows Cyclists to view their personal account. **R5:** The platform allows Cyclists to modify their personal account. **R12:**The platform allows the Cyclists to publish/unpublish a route. **R15:** The platform allows the User to view the details of a path such as the map, the statistics and the status. **R16:** The platform allows the User to visualize the details of the routes related to a path. **R20** The platform merges the status when more than one is present. **R21** The platform compu tes a score for each path based on path's status and average speed.	**D1:** The cyclist's account data is correct. **D2:** The cyclist remembers his credentials. **D9:** All users have a reliable internet connection.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

G4: Given a starting point and a destination, the software suggests the best paths to the user visualizing them in a ranking based on scores and allowing the user to be guided by a navigation system along the selected path.	
R1: The platform allows the Cyclists to create an account. **R2:** The platform allows Cyclists to authenticate. **R3:** The platform allows the access as a Guest. **R8:** The platform allows the Cyclists to confirm and complete the automatically acquired information. **R9:** The platform collects data from the device sensors. **R10:** The platform collects weather data and enriches the path's statistics. **R11:** The platform allows the Cyclists to evaluate the status of a route. **R15:** The platform allows the User to view the details of a path such as the map, the statistics and the status. **R16:** The platform allows the User to visualize the details of the routes related to a path. **R17:** The platform allows the User to search a path by setting the start and the end points with a kilometer tolerance. **R18:** The platform suggests to the User a list of available routes ranked by score. **R19:** bbp calcola il punteggio per ogni percorso (status/tempo impiegato) **R20:** The platform merges the status when more than one is present. **R21:** The platform computes a score for each path based on path's status and average speed.	**D2:** The cyclist remembers his credentials. **D4:** When creating and evaluating a route, the cyclist inputs accurate and realistic information. **D5:** Reliable meteorological information. **D6:** If cyclists use GPS, accelerometer and gyroscope we assume that they are accurate and work correctly **D7:** There are always paths available from point A to point B for any tollerance level. **D8:** The starting and ending point are reasonable, so that they do not necessarily require the use of other means of transportation. **D9:** All users have a reliable internet connection.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

3.3 Performance Requirements

The system must guarantee adequate performance to ensure a smooth and responsive user experience under normal and peak usage conditions.

Core functionalities such as user authentication, route visualization, and path search must respond within few seconds. Automatic data acquisition from mobile sensors must be handled in near real-time, without causing significant delays or excessive battery consumption. The system must be able to support multiple concurrent users, scaling appropriately when there is an increasing number of active cyclists without degradation of service quality.

3.4 Design Constraints

3.4.1 Standards compliance

To conform the system with standards for data protection, the communication must use HTTPS protocol for secure data exchange. Moreover, the data management must adhere to applicable regulations (e.g., GDPR), ensuring protection of users' personal data.

3.4.2 Hardware limitations

The platform is primarily intended for execution on mobile devices, which impose constraints in terms of processing power, memory, storage, and battery capacity. The system must be designed to efficiently use hardware resources, avoiding intensive computations on the mobile devices whenever possible. Access to sensors such as GPS, accelerometer, and gyroscope depends on the availability and accuracy provided by the device hardware.

3.4.3 Other constraints

The system architecture must support interoperability with external services, whose availability and interfaces may impose additional constraints.

3.5 Software System Attribute

3.5.1 Reliability

The system must operate correctly and consistently under normal conditions, minimizing the occurrence of failures. In case of errors or unexpected conditions, the system must handle them rapidly, preventing data corruption and preserving system integrity.

3.5.2 Availability

The system must be available to users for at least 99% of the time, especially during daylight hours and the scheduled maintenance should be planned in the night hours where the number of active users is expected to be lower.

Critical services, such as route creation or path search and navigation, must be designed to minimize downtime and ensure continuity of service without delay.

3.5.3 Security

The system must protect all exchanged and stored data from unauthorized access. Sensitive information, such as user credentials and real-time location data, must be transmitted using secure communication protocols and encrypted when necessary.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

Authentication mechanism must be enforced to ensure that only authorized users can access restricted functionalities and to prevent cyber attacks, such as SQL injection.

3.5.4 Maintainability

The system must be well-documented and designed following modular principles to facilitate maintenance and future enhancements. Changes to one component must have minimal impact on others, allowing easier bug fixing, updates, and future feature implementations.

3.5.5 Portability

The system must be portable across different mobile devices with different operative systems, requiring minimal adaptation of user interfaces. The platform is not intended to be used on desktop devices or in general devices that cannot be carried during a bike ride.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

4 Formal Analysis Using Alloy

4.1 Static Analysis With A Complete Model

In this section there is a formal analysis using Alloy 6. The model captures the core domain concepts (cyclist, path, route) as well as supporting data types (dates, times, locations) and enumeration (status, creationMode, bikeInfrastructure). There are constraints on attribute domains such as valid ranges for scores, dates, time and sensorvalues to have consistent data in our model. There are also fundamental facts that enforce key concept required by the system logic such us uniqueness proprieties and links. There is also specified the rule connecting publication and visibility: a path is public iff contain at least one publish route and a path must contain at least one path.

```
//Status enumeration
abstract sig Status {}

one sig optimal extends Status{}
one sig medium extends Status{}
one sig sufficient extends Status{}
one sig requiresMaintenance extends Status{}
one sig bad extends Status{}

//CreationMode enumeration
abstract sig CreationMode {}
one sig automatic extends CreationMode{}
one sig  manual extends CreationMode{}

//BikeInfrastracture enumeration
abstract sig BikeInfrastructure {}

one sig dedicatedLane extends BikeInfrastructure{}
one sig sharedLine extends BikeInfrastructure{}
one sig mixedTraffic extends BikeInfrastructure{}
one sig noBikeSupport extends BikeInfrastructure{}

//Bool type
abstract sig Bool {}
one sig True extends Bool {}
one sig False extends Bool {}


sig Date {
  day: one Int,
  month: one Int,
  year: one Int
} {
  year >= 0
  month >= 1 and month <= 12
  day >= 1 and day <= 31
}

sig Time {
  hour: one Int,
  minute: one Int,
  second: one Int
} {
  hour >= 0 and hour <= 23
  minute >= 0 and minute <= 59
  second >= 0 and second <= 59
}

sig Location {
  latitude: one Int,
  longitude: one Int
}
```

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
sig Cyclist {
  id: one Int,
  name: one String,
  email: one String,
  password: one String,
  records: set Route
}

sig Path {
  ID: one Int,
  name: one String,
  score: one Int,
  creationDate: one Date,
  averageDuration: one Time,
  streets: some Street,
  currentStatus: one Status,
  isPublic : one Bool
}{
    score >= 0 and score <= 100
}

sig Street {
  name: one String,
  length: one Int,
  trafficLevel: one Int,
  startPoint: one Location,
  endPoint: one Location,
  infrastructureType: one BikeInfrastructure
}

sig Route {
  mode: one CreationMode,
  duration: one Time,
  isPublished: one Bool,
  routeStatus: one Status,
  onPath: one Path,
  sensorSample: lone SensorData,
  routeWeatherCondition: lone WeatherData
}

sig SensorData {
  speed: one Int,
  acceleration: one Int,
  rotation: one Int
}{

  speed >= 0
  acceleration >= 0
  rotation >= 0
}

sig WeatherData {
  condition: one String,
  temperature: one Int
}
```

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
// function that returns all the routes of a path
fun routesOfPath[p: Path] : set Route {
  p.~onPath
}

//  function that returns all the oublic routes of a path
fun publishedRoutesOfPath[p: Path] : set Route {
  { r: routesOfPath[p] | r.isPublished = True }
}


// all the paths have at least one route
fact NoPathsWithoutRoutes {
  all p: Path | some routesOfPath[p]
}

// a path is public if it has at least one public route
fact PathPublicIffSomePublished {
  all p: Path |
    (p.isPublic = True) iff some publishedRoutesOfPath[p]
}



assert Assert1_OneRouteOnePath {
  all r: Route | one r.onPath
}

check Assert1_OneRouteOnePath for 8 but 6 Int


assert Assert2_NoPathsWithoutRoute {
  all p: Path | some routesOfPath[p]
}

check Assert2_NoPathsWithoutRoute for 8 but 6 Int



assert Assert3_PublicPathAtLeastOnePublic {
  all p: Path | (p.isPublic = True) iff some publishedRoutesOfPath[p]
}

check  Assert3_PublicPathAtLeastOnePublic for 8 but 6 Int


assert Assert4_RoutePublicIfPathPublic {
  all r: Route | (r.isPublished = True) implies (r.onPath.isPublic = True)
}

check A4_PublishedRouteImpliesPublicPath for 8 but 6 Int
```

Finally, we validated the correctness of our model writing several assertions and then checking the correctness of our model (all checks do not find any counterexample). In addition, we run a generic predicate, without any constraint, to confirm that the model is satisfiable and that there exist concrete instances that satisfy all facts and constraints.
This step provides evidence that the specification is coherent and that the conceptual model designed for the application can be instantiated consistently.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

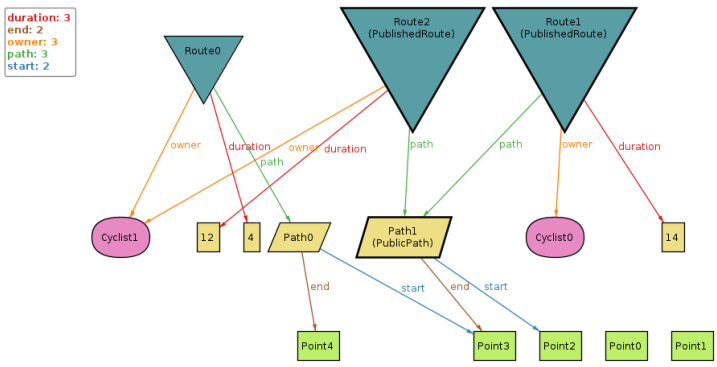
4.2 Static Analysis With A Simplified Model

The analysis continues by introducing a simplified version of the model. The goal of this abstraction is not to change the system requirements, but to make the generated instances easier to read and interpret within Alloy's visualizer. This because the complete model produces a large number of atoms and relations that does not permit a clear visualization of the different behaviors of our model.

```
sig Cyclist {}
sig Point {}

sig Path {
  start: one Point,
  end: one Point
}

sig Route {
  owner: one Cyclist,
  path: one Path,
  duration: one Int
}{
    duration > 0
}

sig PublishedRoute in Route {}
sig PublicPath in Path {}


fun getRoutes[p: Path]: set Route {
p.~path }

fact PublicPathIfAtLeast0nePublicRoute {
  all p: Path | (p in PublicPath) <=> some (getRoutes[p] & PublishedRoute)
}
```

The simplified model keeps only the core concept of Cyclist, Route and Path and all details data are removed. Now a path is represented only by start and end points while a Route keeps just the owner, the path and the duration. The publication concept is modeled as subsets: PublishedRoute and PublicPath.

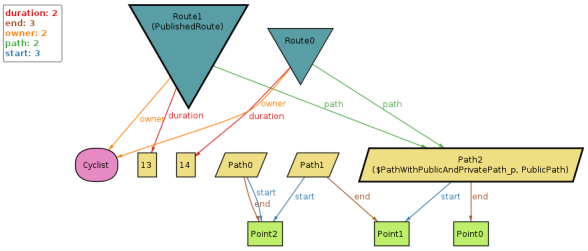
```
pred WorldSample {
  #Cyclist >= 2
  #Path >= 2
  #Route >= 3

  some PublishedRoute
  some (Route - PublishedRoute)
}

run WorldSample for 6 but 5 Int
```



This predicate, WordlSample, shows a generic world with different routes, private and public paths and two cyclists.

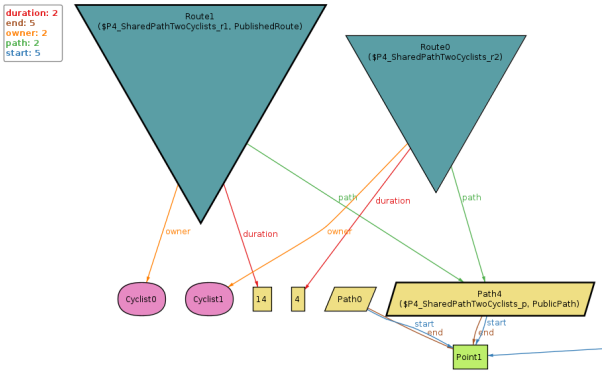
Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
pred PathWithPublicAndPrivatePath {
  some p: Path |
    some (getRoutes[p] & PublishedRoute) and
    some (getRoutes[p] - PublishedRoute)
}

run PathWithPublicAndPrivatePath for 6 but 5 Int
```



This instance shows an important property of the system. If a path contains at least one published route, then the path is public even if it also contains private routes. In BPP, only published routes are visible to other users but private routes are still linked to the same path at the data level. This avoids creating a separate "private copy" of the same path. In this way if a cyclist decides to publish a previously private route it will automatically belong to the correct existing public path, preventing duplicates of the same path in the system.

```
pred SharedPathTwoCyclists {
  some p: Path |
    some disj r1, r2: getRoutes[p] |
      r1.owner != r2.owner and
      some (getRoutes[p] & PublishedRoute)
}

run SharedPathTwoCyclists for 6 but 5 Int
```



This predicate shows the community aspect of BPP where a single path can aggregate multiple routes performed by different Cyclists.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

4.3 Dynamic Analysis

Dynamic analysis helps to understand how model entities evolve over key operations and is performed on a simplified model, consistent with the approach adopted for static analysis. State changes and system behavior over time were described using the built-in support provided by Alloy 6.

```
sig Cyclist {}

sig Point {}

sig Path {
  start: one Point,
  end: one Point
}

sig Route {
  owner: one Cyclist,
  var onPath: lone Path
}

var sig PublishedRoute in Route {}
var sig PublicPath in Path {}
var sig ActivePath in Path {}
var sig ActiveRoute in Route {}

fun routesOf[p: Path] : set Route {
 p.~onPath
}

fact Init {
  no onPath
  no PublishedRoute
}


fact DerivedSets {
  always ActiveRoute = onPath.Path
  always ActivePath  = Route.onPath
  always PublicPath  = { p: Path | some (routesOf[p] & PublishedRoute) }
  always PublishedRoute in ActiveRoute
}
```

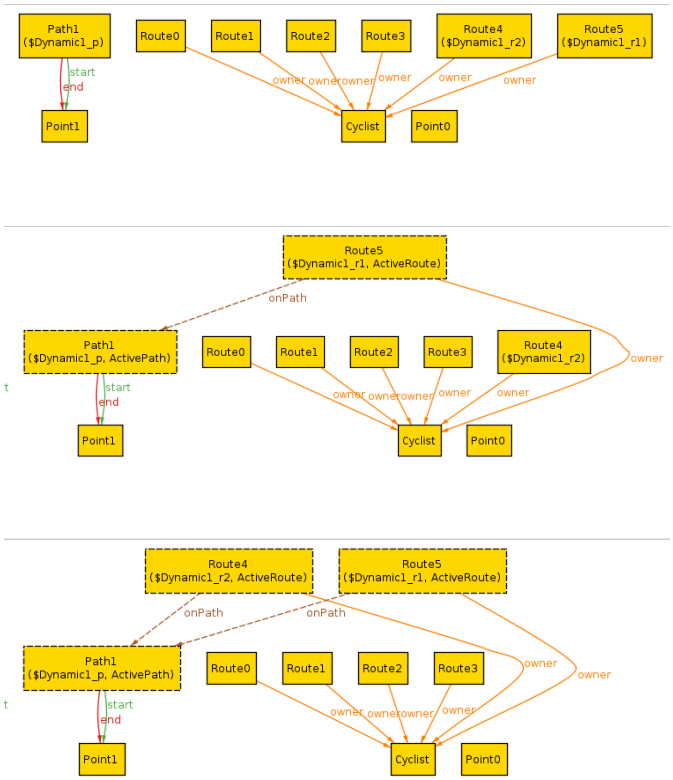
Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
fact DerivedSets {
  always ActiveRoute = onPath.Path
  always ActivePath  = Route.onPath
  always PublicPath  = { p: Path | some (routesOf[p] & PublishedRoute) }
  always PublishedRoute in ActiveRoute
}

pred Stutter {
  onPath' = onPath
  PublishedRoute' = PublishedRoute
}

pred CreateNew[r: Route, p: Path] {
  no r.onPath
  no routesOf[p]
  onPath' = onPath ++ r->p
  PublishedRoute' = PublishedRoute - r
}

pred CreateExisting[r: Route, p: Path] {
  no r.onPath
  some routesOf[p]
  onPath' = onPath ++ r->p
  PublishedRoute' = PublishedRoute - r
}

pred Publish[r: Route] {
  some r.onPath
  r not in PublishedRoute
  onPath' = onPath
  PublishedRoute' = PublishedRoute + r
}

pred Unpublish[r: Route] {
  r in PublishedRoute
  onPath' = onPath
  PublishedRoute' = PublishedRoute - r
}

pred Delete[r: Route] {
  some r.onPath
  onPath' = onPath - r->Path
  PublishedRoute' = PublishedRoute - r
}


pred Step {
  Stutter
  or some r: Route, p: Path | CreateNew[r,p] or CreateExisting[r,p]
  or some r: Route | Publish[r] or Unpublish[r] or Delete[r]
}

fact Trace {
 always Step
}
```

The dynamic model represents the system evolution through a mutable relation onPath and a mutable set PublishedRoute. It is forced an initial state, Init. The predicates CreateNew, CreateExisting, Publish, Unpublish, Delete model the key operations of the BPP.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

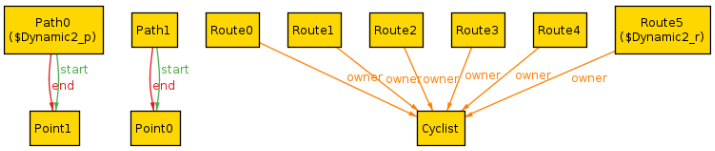
```
//Create a route with a new path and then associate another route on the path
pred Dynamic1 {
  some disj r1, r2: Route, p: Path |
    ( CreateNew[r1,p] ;
      CreateExisting[r2,p] ;
      (r1 in routesOf[p] and r2 in routesOf[p])
    )
}

run Dynamic1 for 6 Route, 4 Path, 4 Cyclist, 6 Point, 4 steps
```

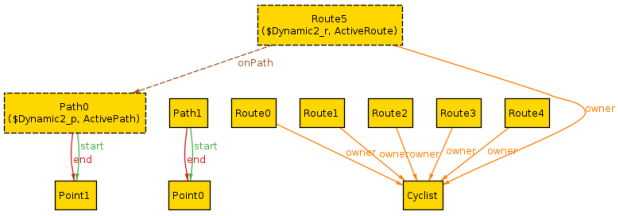


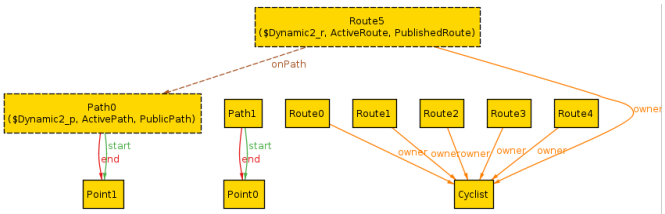
This predicate shows a trace in which a first route is recorded and a new path is created (activated). Additional routes are then added and linked to the same path. The key point is that the system reuses the existing path instead of creating duplicates.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
//A route initially private and then public
pred Dynamic2 {
  some r: Route, p: Path |
    ( CreateNew[r,p] ;
      (p not in PublicPath and Publish[r]) ;
      (p in PublicPath)
    )
}

run Dynamic2 for 6 Route, 4 Path, 4 Cyclist, 6 Point, 4 steps
```



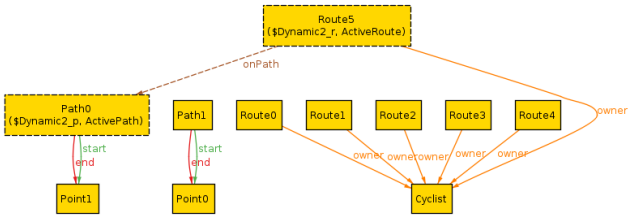


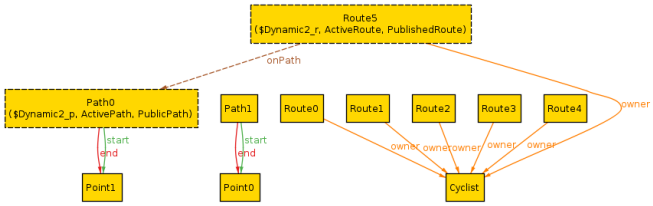


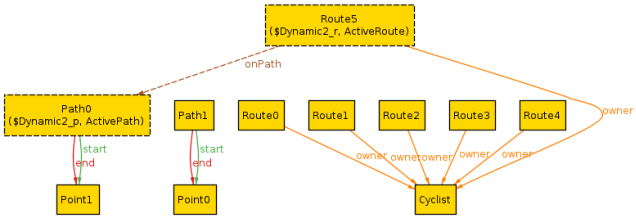
This predicate shows a route that is initially private and then becomes published. After the publish operation, the route is added to PublishedRoute and its associated path becomes part of PublicPath.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
// Last public route of a path is unpublished then the path becomes unpubblish
pred Dynamic3 {
  some r: Route, p: Path |
    ( CreateNew[r,p] ;
      Publish[r] ;
      (no ((routesOf[p] & PublishedRoute) - r) and Unpublish[r]) ;
      (p not in PublicPath)
    )
}

run Dynamic3 for 6 Route, 4 Path, 4 Cyclist, 6 Point, 4 steps
```



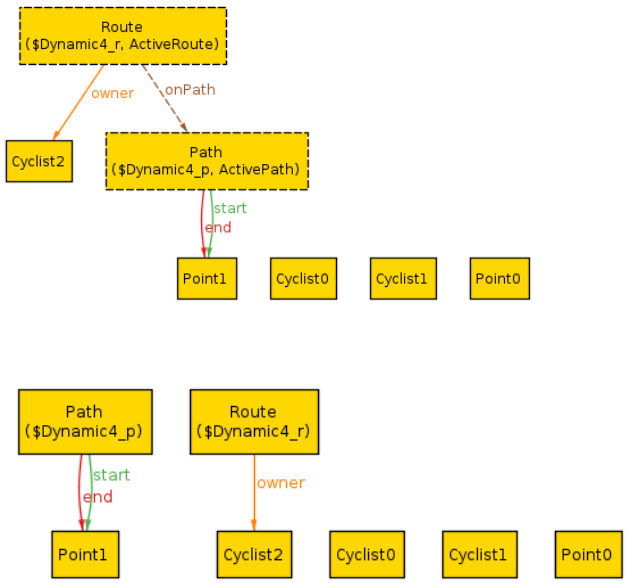




This is the opposite situation of the previous case. Initially, a route is published, making its path public. When the route is later unpublished and the path has no remaining published routes, the path is no more publish.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

```
// Last route of a path removed so the path became inactive
pred Dynamic4 {
  some r: Route, p: Path |
    ( CreateNew[r,p] ;
      (routesOf[p] = r and Delete[r]) ;
      (p not in ActivePath)
    )
}

run Dynamic4 for 6 Route, 4 Path, 4 Cyclist, 6 Point, 4 steps
```



In this case, a path is created with a single route, and then the last route is removed. Since the path no longer contains any routes, it is considered inactive, modeling the cleanup of an empty path.

4.4 Conclusions from Alloy Model Analysis

The alloy analysis provides a clear formal validation of the application model. Through static analysis, the main entities, relationships and global constraints were checked for consistency, demonstrating that valid instances are admitted.
Dynamic analysis models the key operations over time showing how routes and paths evolve through creation, publication, and deletion. These results clarify the intended behavior of the system and offer strong guidance for implementing the application without inconsistencies.

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

5 Effort Spent

	Chapter 1	Chapter 2	Chapter 3	Chapter 4
Flavio	3	8	19	8
Salvatore	4	10	23	1
Anna	4	11	20	3

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved

6 Reference

The softwares that have been used are:

- **Astah UML** has been used for Sequence diagrams and Activity diagrams
- **Alloy 6.2.0 Visualizer and Compiler** has been used for alloy analysis
- **www.lovable.dev** has been used to create UI mockups
- **www.overleaf.com** has been used as a collaborative writing tool to create the document

Copyright © 2025, Bellomo Flavio, Pennino Salvatore, Samarani Anna – All rights reserved